10. Úkol 6: Implementace řídicího SW pro PLC

Cílem úkolu je samostatně v jednotlivých skupinách vytvořit PLC program, který je modulárně strukturován v souladu s normou S88 pro dávkovou výrobu.

10.1. Pokyny k vypracování

1. V inženýrském rámci TwinCAT vytvořte obslužný program pro navrhovanou technologii tak, aby splňovaly požadavky pro připojení k systému SkuBatch a dávkového řízení. Těmito požadavky jsou:
    a. Možnost řídit jednotlivá řízená zařízení (ventily, míchadla, ohřev) manuálně prostřednictvím vizualizace.
    b. Možnost aktivovat automatické řízení jednotlivých zařízení prostřednictvím fází.
    c. Možnost řídit fáze v PLC (napouštění, vypouštění, míchání, ohřev) v manuálním režimu prostřednictvím vizualizace.
    d. Možnost aktivovat automatické řízení jednotlivých fází v PLC prostřednictvím fází v MES systému **SkuBatch.**
2. Pro tvorbu programu využijte předpřipravenou šablonu, která se nachází v kartě předmětu.
3. Jako vstupy a výstupy použijte vstupy a výstupy softwarového simulátoru
4. Detailní požadavky na SW jsou umístěny v následujícím textu.

10.2. Požadovaný výstup

Požadovaným výstupem, který bude předveden vyučujícímu je:

1. Funkčnost jednotlivých řídicích modulů
2. Funkčnost jednotlivých modulů zařízení
3. Funkčnost rozhraní pro systém SkuBatch

Do dokumentace, vytvořené v rámci předchozích cvičení doplníte:

1. Popis PLC programu – jednotlivých bloků + celkové logiky funkce
2. Popis postupu návrhu regulátoru
3. Manuál – návod k použití (může být také do samostatného dokumentu) – kde popíšete vizualizaci a její ovládání pro operátora technologie

K odevzdání dokumentace přiložte také kompletní program (zip)

10.3. Programová kostra

Cílem cvičení je ukázat studentům, jakým způsobem vytvořit PLC program tak, aby byl srozumitelný, udržovatelný a v souladu s danými požadavky. Za tímto účelem je v informačním systému v kartě předmětu připravena kostra programu, která tyto požadavky splňuje. Není bezpodmínečně nutné z této kostry vycházet, nicméně pokud se rozhodnete pro vlastní implementaci, ***musíte zachovat všechny výše uvedené požadavky.***

Předpřipravená kostra programu obsahuje HW konfiguraci, seznam obecných a vstupně výstupních proměnných (mapování vstupů a výstupů na simulátor) a strukturu objektů (funkce, datové typy, datové objekty).

10.3.1. Přehled kostry

Před začátkem práce je vhodné projít a zkontrolovat si jednotlivé části kostry programu:

- Povšimněte si, že každý z objektů je určitého datového typu a podívejte se, jak jsou pro jednotlivé skupiny objektu tyto typy opakovaně využívány.
- Rozbalte položku DUTs v PLC projektu a projděte si jednotlivé vytvořené datové typy. Všimněte si, jak jsou do sebe jednotlivé struktury zanořeny. Zorientujte se, které datové typy popisují řídicí moduly, a které pak fáze.
- Rozbalte položku POUs a otevřete si postupně bloky Main_Batch, FB_Tank, bloky řídicích modulů (pojmenovány podstatnými jmény - Heater), bloky fází (pojmenovány také podstatnými jmény, ale označujícími činnosti - Heating). Všechny bloky si pořádně prohlédněte. Pokud nebudete něčemu rozumět, zeptejte se!
- Věnujte alespoň půl hodiny tomu, že si projdete strukturu programu tak, abyste při následujícím čtení nemuseli vždy dlouho hledat požadované entity. Je doporučeno psát si poznámky.

10.3.2. Základní stavební bloky programu

Následující kapitola jednotlivě popisuje základní stavební bloky kostry programu.

10.3.2.1. MB_Connector

Programový blok, ve kterém je implementována výměna dat se simulátorem. Vyčtená data ze simulátoru jsou vkládána do globálních proměnných s prefixy **di_** a **ai_**, data, která budou zapisována do simulátoru jsou brána z globálních proměnných s prefixy **do_** a **ao_**.

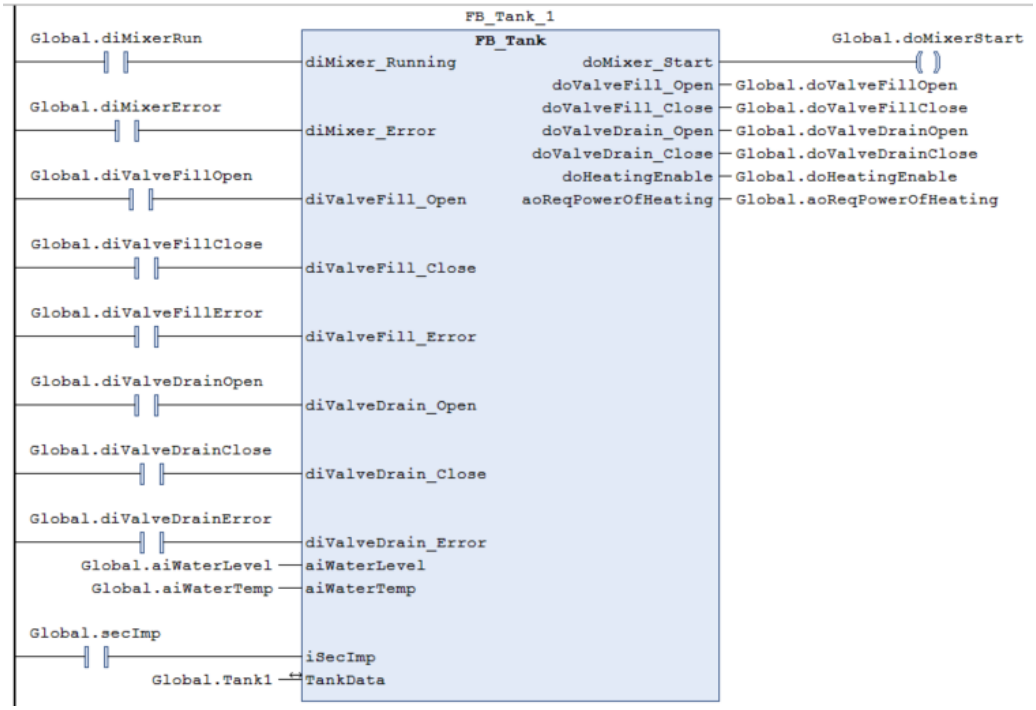
10.3.2.2. Main_Batch

Programový blok **Main_Batch**, jehož volání je zajištěno pomocí PLC Tasku **Default** volá cyklicky funkční blok typu ***FB_Tank*** instanciovaný jako **FB_Tank_1**. Tato funkce obsahuje ostatní potřebné funkce a funkční bloky pro kompletní funkcionalitu řízení jednoho tanku.

Povšimněte si, že vstup/výstupy do/z tanku jsou přímo binární či analogové proměnné z technologického procesu (mohly by to být přímo fyzické vstup/výstupy, nicméně ty my nemáme, místo toho jsou jimi bity získané parsováním komunikačního rámce).

Všimněte si také velmi důležité vstupně/výstupní proměnné **Tank1** (poslední na levé straně, označená dvojšipkou), která předává kompletní instanci globálního datového bloku Tank1 typu **TTank** do řídicí struktury.

```
PROGRAM Main_Batch
VAR
    FB_Tank_1 : FB_Tank;
END_VAR
```



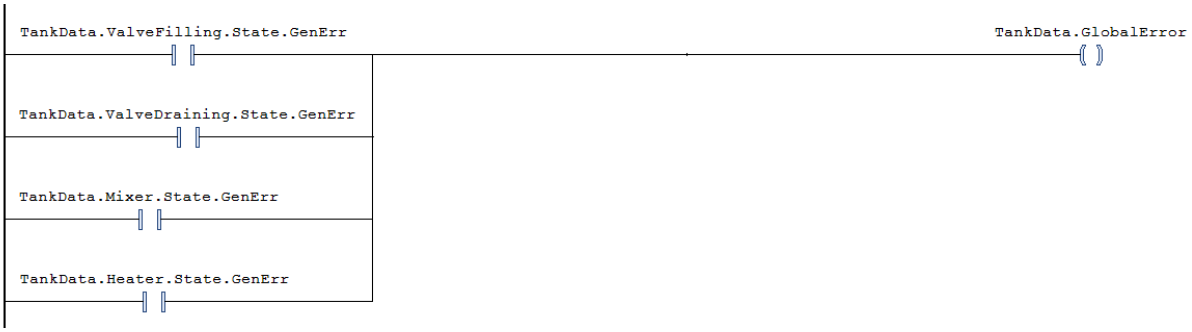
Obrázek 36 Volání funkčního bloku tank v programu Main_Batch

Instanciací funkčního bloku FB_Tank (např. FB_Tank_2) s novou instancí datového bloku a namapováním jiných vstupně/výstupních proměnných na nožičky této funkce (z další jednotky – fyzického tanku) je pak jednoduché vytvořit řízení většího množství tanků. Pro každou instanci funkčního bloku by bylo potřeba vyrobit a konfigurovat také instanci datového bloku.

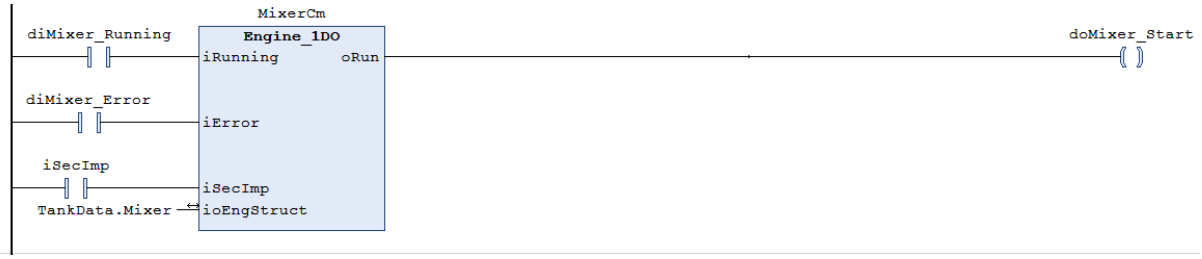
Funkční blok typu FB_Tank tedy obsahuje veškeré funkcionality pro řízení jednoho tanku, těmi jsou:

- řízení akčních členů prostřednictvím volání bloků modulů zařízení - 2x ventil, motor a topení,
- řízení fází - funkce pro napouštění, vypouštění, míchání a zahřívání,
- různé obslužné rutiny triviálního rázu.

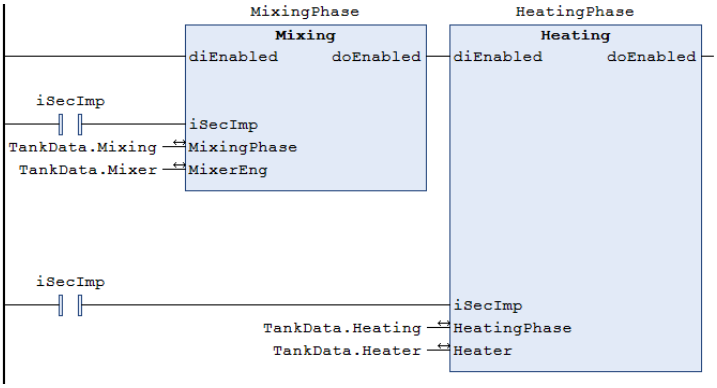
Ze vstupních proměnných ***FB_Tank*** se uvnitř předávají jednotlivé proměnné do daných funkcí a výstupy se přiřazují na výstup funkčního bloku ***FB_Tank***.



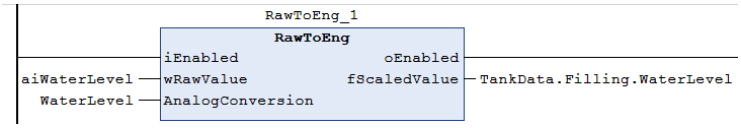
Obrázek 37 Logický součet dílčích chybových příznaků jednotlivých modulů zařízení udávající příznak GlobalError využívaný v rámci vizualizace i komunikace s nadřazeným systémem



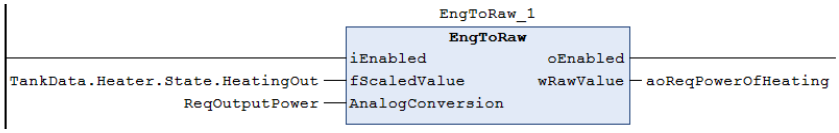
Obrázek 38 Volání funkčního bloku řídicího modulu mísení – vidíme zde přenesení vstupních parametrů bloku FB_Tank (jako je diMixer_Running) dovnitř bloku Mixer (jehož konkrétní instance je součástí datového bloku TankData předaného do FB_Tank zvnějšku). Stejným způsobem se volají další řídicí moduly



Obrázek 39 Volání funkčního bloku fáze míchání – opět vidíme jak propojení vstupních parametrů bloku FB_Tank s parametry instance funkčního bloku MixingPhase. Těmi jsou datový blok fáze a datový blok konkrétního vypouštěcího ventilu. Vstupní sekundový impulz slouží k realizaci různých funkcí v rámci funkčního bloku. Stejným způsobem jsou volány všechny fáze

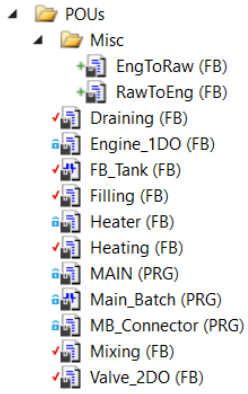


Obrázek 40 Funkční blok pro přepočet dat z převodníku (komunikace) na inženýrské hodnoty. Povšimněte si, že parametry převodu jsou schovány ve struktuře WaterLevel, která je součástí definice funkčního bloku tanku. V případě více tanků s různými potřebnými parametry bychom museli nastavení parametrů vyčlenit z FB_Tank na jiné místo.



Obrázek 41 Funkční blok pro přepočet dat z inženýrské hodnoty na data pro převodník (komunikaci). Opět platí to stejné, co bylo napsáno o opačné funkci RawToEng.

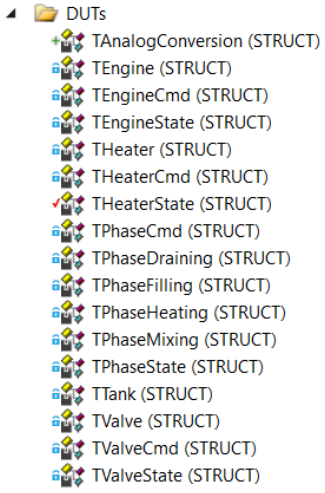
Všechny výše zmíněné funkční bloky, jak je možné po otevření vidět, jsou prázdné, nebo obsahují pouze komentáře, které popisují funkcionalitu jednotlivých částí kódu, které mají být Vámi dopsány.



Obrázek 42 Funkce pro jednotlivé celky

10.3.3. Uživatelsky definované datové typy kostry

Uživatelsky definované struktury - typy (DUT - user data types) jsou již v kostře programu předdefinovány. Vaše inovace je možná v DUT pro zařízení, nicméně pro splnění požadavků na implementaci plně dostačuje využití navrhnutých v předpřipraveném projektu. ***Z důvodu propojení dat pomocí názvu mezi PLC a MES systémem (pomocí protokolu OPC UA), není možné zasahovat do struktur DUT fází (tedy těch s prefixem TPhase).***



Obrázek 43 Typy datových struktur

Hierarchie datových typů je následující:

- Datové typy pro řídící moduly jsou Engine, Heater a Valve a obsahují struktury State (tato struktura kompletně popisuje stav řídicího modulu), Cmd (tato struktura kompletně specifikuje příkazy, pomocí kterých může být modul ovládán ve všech režimech) a triviální datové typy, popisující vstupní/výstupní parametry.

  Příkladem může být datový blok Heater, jehož vstupem je požadovaná aktuální hodnota teploty (iWaterTemp), výstupem pak akční zásah (oHeaterPower) a binární informace o spuštění topného elementu. Setpoint teploty (nastavuje se "ručně") a výkon topení v manuálním režimu (opět ruční nastavení) jsou schovány ve struktuře ioHeatStruct, která je do funkčního bloku předána z nadřazeného funkčního bloku FB_Tank. Tato struktura je vstupně/výstupním parametrem, tedy jakékoli změny budou udělány, promítnou se do "hlavní" instance (Tank1 v

programu Main_Batch). Vnitřní parametry, jako je např. pidMode slouží pro využití výhradně během volání funkčního bloku.

```
FUNCTION_BLOCK Heater
VAR_INPUT
    iWaterTemp : REAL;
END_VAR
VAR_OUTPUT
    oHeaterEnable : BOOL;
    oHeaterPower : REAL;
END_VAR
VAR_IN_OUT
    ioHeatStruct : THeater;
END_VAR

VAR
    pidParameters : ST_CTRL_PID_PARAMS;
END_VAR
```

- Datové typy pro fáze jsou označeny Phase… a obsahují rovněž struktury State, Cmd (Commands) a triviální datové typy sloužící pro nastavení parametrů.
- Datový typ tanku obsahuje 2 x UDT Valve (napouštěcí, vypouštěcí ventil), 1 x Engine (míchadlo) a 1 x Heater (ohřev). Dále obsahuje datový blok pro každou fázi (PhaseFilling, PhaseDraining, PhaseHeating, PhaseMixing).
- Odlišný **typ** tanku by mohl obsahovat jiné akční elementy a případně i fáze
- Stejný typ tanku - druhý tank - může využít celý datový typ bez modifikace

10.3.4. Popis koster řídicích modulů

Implementace řídicího modulu bude popsána na modulu Engine_1DO, který je využit pro řízení míchadla (a může být v budoucnu použit pro řízení jakéhokoli řízeného zařízení, které se ovládá přepínáním jedné své vstupní hodnoty - Zapnuto/Vypnuto a svůj chod signalizuje také jedním svým výstupem). V příslušném funkčním bloku se lze podívat na jeho vstupy a výstupy.

```
FUNCTION_BLOCK Engine_1DO
VAR_INPUT
    iRunning : BOOL;
    iError : BOOL;
    iSecImp : BOOL;
END_VAR
VAR_OUTPUT
    oRun : BOOL;
END_VAR
VAR_IN_OUT
    ioEngStruct : TEngine;
END_VAR
VAR
    secImpEdge: BOOL;
    tempStartupFail: BOOL;
END_VAR
```

Obrázek 44 Vstupy a výstupy funkčního bloku

Vstupními signály jsou:

- iRunning - zpětná vazba o tom, zda pohon běží (běžně se používá pouze zpětná vazba od stykače, proudové/napěťové relé, snímač otáček nebo zpětná vazba z FM).
- iError – signalizace chyby pohonu. Signalizace bývá z FM, tepelné ochrany pohonu, zařízení hlídající prosak vody do pohonu/oleje apod.

- iSecImp - vstup sekundového impulzu pro počítání motohodin. V demonstrační úloze budeme zobrazovat motominuty/motosekundy (záleží na Vás), v běžné praxi stačí pro obsluhu zobrazovat pouze motohodiny.

Výstup je zde jediný a slouží pro ovládání digitálního výstupu, kterým se spíná pohon (většinou stykač (pro malá zařízení relé), signál pro frekvenční měnič atp.).

Jako jediný InOut (data, která jsou definována jako výstup, ale je možné je číst a zapisovat/měnit je) je struktura ioEngStruct datového typu Engine. Jak bylo již zmíněno výše, datové struktury jsou předdefinovány a je vhodné se na ně podívat a ujasnit si význam jednotlivých proměnných. Uvnitř této struktury jsou mezi jednotlivými voláními bloku udržovány hodnoty všech jeho vnitřních stavů (proměnných)

Expression	Type	Value
iRunning	BOOL	FALSE
iError	BOOL	FALSE
iSecImp	BOOL	FALSE
oRun	BOOL	FALSE
ioEngStruct	REFERENCE TO...	
Cmd	TEngineCmd	
SetRT	DINT	<Dereferenc...
AutCtrl	BOOL	<Dereferenc...
ManCtrl	BOOL	<Dereferenc...
TurnOnMan	BOOL	<Dereferenc...
TurnOffMan	BOOL	<Dereferenc...
Req2RunAutCtrl	BOOL	<Dereferenc...
ResetSUF	BOOL	<Dereferenc...
State	TEngineState	
RT	DINT	<Dereferenc...
Run	BOOL	<Dereferenc...
RunOut	BOOL	<Dereferenc...
Err	BOOL	<Dereferenc...
StartUpFail	BOOL	<Dereferenc...
AutCtrl	BOOL	<Dereferenc...
ManCtrl	BOOL	<Dereferenc...
MayRun	BOOL	<Dereferenc...
MayAutoRun	BOOL	<Dereferenc...
GenErr	BOOL	<Dereferenc...
StateInt	INT	<Dereferenc...
StartupFailTime	INT	<Dereferenc...
Time2Start	INT	<Dereferenc...
secImpEdge	BOOL	FALSE
tempStartupFail	BOOL	FALSE

Obrázek 45 Paměťová struktura motoru

Každá struktura pro ovládání v kostře programu vypadá podobně jako ***DUT Engine***. Skládá se ze tří základních částí: příkazy (Cmd), stavy (State) a parametry (Param).

Jednotlivými stavy jsou:

- ***RT (Running Time)*** - motohodiny, datový typ Double Integer dostačuje pro počítání motohodin, z vlastní zkušenosti (TB) integer nestačí)
- ***Run*** - bool, signalizace chodu pohonu, pokud vstupní signál signalizuje chod, tento bit má hodnotu true.
- ***RunOut*** - bool, signál slouží pro detekci toho, zda se funkční blok snaží sepnout výstup. Pokud bychom ve funkci chtěli číst výstup, bude to možné, ale taková část kódu bude označena jako nesyntakticky správně (program fungovat bude, ale tento řádek bude podtrhnutý žlutě). Hrozí

zde nebezpečí, že hodnotu přečteme špatně - sepneme na jednom řádku výstup a v následujícím ho čteme, bohužel zde nebude výstup ještě sepnutý, záleží na tom, kdy PLC své výstupy aktualizuje. Zpravidla jako defaultní nastavení to bývá na konci cyklu smyčky, je možné jej dělat i periodicky, ale také (při použití distribuovaných I/O) to může být synchronně v závislosti na použité komunikaci nebo vzorkovací periodě sběrnice I/O karet.

- ***Err*** - Stejně jako signál Run, jedná se o přepis vstupní proměnné iErr.
- ***StartUpFail*** - viz požadavky na řízení zařízení *Nerozběh*. Pokud sepneme pohon do určité doby, očekáváme informaci o chodu. S tím souvisí parametr Time2Start v UDT Engine a také nadefinovaný časovač SUF_Timer ve funkčním bloku.
- ***AutCtrl/ManCtrl*** (automatické/ manuální ovládání), pohon musí být v jednom z těchto dvou stavů, pokud je v režimu manuálním, je možné ho ovládat povely z HMI/SCADA, pomocí Cmd TurnOnMan, TurnOffMan, pokud je v automatickém režimu, pohon je ovládán z PLC pomocí signálu Req2RunAutCtrl.
- ***MayRun/MayAutoRun*** - signalizace vhodná pro prvotní kontrolu zařízení. Pokud je sepnutý MayRun, pak je pohon možné ovládat v manuálním režimu a nemá žádnou poruchu, pokud je aktivní MayAutoRun, pohon nemá žádnou poruchu a je možné jej ovládat
- ***GenErr - (General error)*** většina zařízení má více typů poruchy, signály Err a StartUpFail jsou vhodné pro zápis do alarmového deníku, ale pro signalizaci pohonu červeně ve vizualizaci je vhodné mít sdružený bit poruchy. Poruchu GenErr není potřeba kvitovat zvlášť, zmizí, pokud nebude aktivní žádná z "podřízených" poruch.

Následující seznam popisuje jednotlivé příkazy. Tyto příkazy by měly být vždy po zpracování navráceny do klidové úrovně.

- ***SetRT*** - nastavení motohodin. Jak bylo zmíněno výše, ne vždy je pohon nový, nebo s výměnou řízení přichází i výměna pohonů, je tedy nutné mít možnost nastavovat motohodiny.

- ***AutCtrl/ManCtrl*** - přepínání mezi automatickým a manuálním režimem. V případě přijetí obou signálů zároveň by měl být, z hlediska bezpečnosti upřednostněn manuální režim.

- ***TurnOnMan/TurnOffMan*** - jak bylo již zmíněno, signály slouží pro ovládání pohonu v manuálním režimu, pokud je pohon v automatu, i přesto se signál musí vyhodnotit, akce se nesmí vykonat a signál se vrátí do klidového stavu.

- ***Req2RunAutCtrl*** - signál, kterým PLC ovládá modul zařízení, pokud je v automatu. Tzn. řízení přichází z algoritmu ovládající tento modul zařízení nebo z nadřazeného řízení.

- ***ResetSUF*** - resetování poruchy nerozběhu. Pokud se porucha nerozběhu vygeneruje, zařízení bude v poruchovém stavu. Je na obsluze/operátorovi/údržbě, aby zařízení zkontrolovala a pak tuto poruchu resetovala. (Volitelně) je možné přidání počtu automatických pokusů a intervalu pokusů opakovat pokus rozběhnout pohon automaticky, a až následně bude porucha trvalá a bude třeba vyčkat na obsluhu. Toto je vhodné implementovat např. u ventilů nebo čerpadel apod., kdy je možné, že při delším nepoužívání nebo u starších pohonů kde mohou být mechanické části zatuhlé. Pak se zařízení může chovat následovně: čerpadlo je přicpané nečistotami, modul zařízení se pokusí pohon spustit, po 10s nepřijde zpětná vazba o chodu a pohon se vypne, nečistoty však postoupily o kousek dál. Čerpadlo využívá protékající vodu jako vlastní chlazení motoru, ale na sucho nějaký minimální čas vydrží běžet. Následuje interval, kdy se čeká na automatické opakování (motor chladne - neprotékala voda) a následně se modul zařízení (pokud je stále požadavek na chod) pokusí pohon opětovně sepnout.

10.3.5. Implementace funkcionalit modulů zařízení

Ve funkčním bloku Eng_1DO je připraveno několik částí (regionů). Každá z nich je pojmenovaná a u každé je napsán komentář. Do každé z těchto částí napište pouze kód, který jí přísluší podle konkrétního komentáře.

Tabulka 2 Popis jednotlivých regionů modulu zařízení Eng_1DO

Region	Popis
RunningTime	Použijte vstup iSecImp k navyšování počtu motosekund. Tento vstup má periodu 1 sekundu se střídou 50:50. Buď využijte funkční blok R_TRIG k detekci náběžné hrany, nebo vymyslete mechanizmus jinak. V každém případě budete muset doplnit nějakou "paměťovou buňku" do struktury TEngine. Pokud je hodnota SetRT větší nebo rovna 0, nastavte počet motosekund na tuto hodnotu a hodnotu SetRT nastavte na hodnotu -1. Takto je realizováno uživatelské nastavení hodnoty motosekund.
RunState	Zda zařízení skutečně běží, není možné určit pouze tím, že jsme zadali příkaz k jeho sepnutí. Do stavu Run tedy nastavte hodnotu vyčtenou ze zpětné vazby od zařízení (iRunning).
Manual/Automat	Na základě požadavku na automatický (příkaz Cmd.AutCtrl) nebo manuální režim se změní stavové bity automatického (State.AutCtrl) a manuálního režimu. Nezapomeňte vždy požadavek vyresetovat! Zde rovněž proveďte inizializaci – pokud nebude aktivní automatický ani manuální režim, preferujte manuální.
MR/MAR	Nastavení pomocných příznaků MayRun a MayAutoRun. MayRun je aktivní v případě, že je aktivní manuální režim a není nastaven příznak poruchy. MayAutoRun funguje analogicky pro automatický režim.
ManCtrl	Zpracování příkazů Cmd.TurnOnMan a Cmd.TurnOffMan. Pokud je aktivní příkaz TurnOnMan a současně je aktivní pomocný příznak pro manuální režim, aktivujte stav RunOut. Analogicky postupujte pro příkaz TurnOffMan. Nezapomeňte oba příkazy vynulovat!
AutCtrl	Pokud je aktivní automatický režim a je aktivní příznak MayAutoRun, otestujte příkaz Req2RunAutCtrl. Na základě jeho hodnoty nastavte opět stav RunOut.
ErrStates	1. Vytvořte funkční blok TON časovače (opět je potřeba přidat jeho definici do datového typu TEngine - časovač potřebuje paměť, která se bude nacházet v instanci struktury celého tanku). Po sepnutí výstupu počítá dobu Tim2Start. Pokud do této doby není aktivní zpětná vazba od zařízení, vygeneruje se příznak (stav) StartUpFail.

	2. Dále pokud zařízení aktivovalo svůj výstup iError (vstup do PLC), nastavte příznak (stav) Err. 3. Logickým součtem obou výše popsaných chybových příznaků je příznak GenErr. 4. Příkaz ResetSUF resetuje příznak StartUpFail. Nezapomeňte vyresetovat i tento příkaz! 5. Pokud je aktivní příznak GenErr, je nutné vypnout výstup modulu (deaktivovat zařízení - přepnout do bezpečného stavu) - RunOut. **Za blok ErrStates vložte příkaz, který zrcadlí hodnotu stavu RunOut z datového bloku na výstup #oRun. Hodnoty jsou duplikovány, protože zatímco RunOut je možné i číst a je využívána ve vizualizaci, výstup oRun je připojen přes výstup funkčního bloku tanku přímo na fyzický výstup PLC a ovládá zařízení.**
StatusInt	Pro potřeby vizualizace (přebarvování stavových symbolů) je nutné nastavit hodnotu int proměnné State.StateInt následovně: 1. Pokud není GenErr a modul je spuštěn (ve stavu Run), přiřaďte konstantu #Running (0) 2. Pokud není GenErr a modul je zastaven, přiřaďte konstantu #Stop (1) 3. Pokud je GenErr, nastavte na základě stavu Run konstanty #RunningAndErr (2) nebo pouze #Err (3).

Tabulka 3 Popis jednotlivých regionů modulu zařízení ventilu

Region	Popis
RunningTime	Tento region se zde nevyskytuje, tedy nepočítáme motohodiny servomotoru manipulujícího s ventilem.
RunState	Zda je ventil otevřen či uzavřen není možné určit pouze tím, že jsme zadali příslušný příkaz. Do stavu Open/Closed tedy nastavte hodnotu vyčtenou ze zpětné vazby od zařízení (iOpen, ...).
Manual/Automat	Funkcionalita je totožná s modulem ENG_1DO
MR/MAR	Funkcionalita je totožná s modulem ENG_1DO
ManCtrl	Zpracování příkazů Cmd.OpenMan, Cmd.CloseMan a Cmd.StopMan. Pokud je aktivní příkaz OpenMan a současně je aktivní pomocný příznak pro manuální režim, aktivujte stav OpenOut a deaktivujte stav CloseOut.

	Analogicky postupujte pro příkaz CloseMan. Příkaz StopMan pak deaktivuje oba stavy OpenOut i CloseOut. Nezapomeňte všechny příkazy nakonec vynulovat!
AutCtrl	Pokud je aktivní automatický režim a je aktivní příznak MayAutoRun: 1. Otestujte, zda je nastaven příkaz Req2OpenAut a deaktivován příkaz Req2CloseAut. Pokud je podmínka splněna, aktivujte OpenOut a deaktivujte CloseOut. 2. Pokud jsou příkazy Req2OpenAut a Req2CloseAut prohozeny, přepněte stavy OpenOut a CloseOut obráceně. 3. Pokud není aktivní ani jeden z příznaků Req2OpenAut a Req2CloseAut, deaktivujte oba stavy.
ErrStates	1. Definujte dva TON časovače analogicky, jako v bloku Engine_1DO (a typu TEngine). První: Pokud je sepnut OpenOut a rozepnut CloseOut, počítá dobu Time2Open. Druhý: při obráceném smyslu stavů počítá dobu Time2Close. 2. Pokud do této doby není aktivní zpětná vazba od zařízení, vygeneruje se příznak (stav) StartUpFail. 3. Logický součet výstupů (Q) těchto časovačů dává hodnotu StartUpFail. 4. Dále pokud zařízení aktivovalo svůj výstup iError (vstup do PLC), nastavte příznak (stav) Err. 5. Logickým součtem obou výše popsaných chybových příznaků je příznak GenErr. 6. Příkaz ResetSUF resetuje příznak StartUpFail. Nezapomeňte vyresetovat i tento příkaz! 7. Pokud je aktivní příznak GenErr, je nutné vypnout výstupy modulu (deaktivovat zařízení - přepnout do bezpečného stavu) - OpenOut a CloseOut. Za blok ErrStates vložte příkaz, který zrcadlí hodnotu stavů OpenOut a CloseOut z datového bloku na výstup oOpen a oClose. Hodnoty jsou duplikovány, protože zatímco OpenOut a CloseOut je možné i číst a je využívána ve vizualizaci, výstupy oOpen a oClose jsou připojeny přes výstup funkčního bloku tanku přímo na fyzické výstupy PLC a ovládají zařízení.
StatusInt	Opět pro potřeby vizualizace je nutné nastavit int hodnotu StColor. Vzhledem k množství možných hodnot máte tento blok předvyplněn.

Pro ventily (Val_2DO) budou existovat dva výstupy - pokyn pro zavření a pokyn pro otevření. Tyto dva výstupy by neměly nikdy být sepnuty zároveň. Struktura funkčního bloku je velice podobná, nezapomeňte vždy změnit zejména název interního datového bloku na iVal_Struct.

Pro ohřev (Heater) budeme mít opět pouze jeden číslicový výstup a jednu zpětnou vazbu ze zařízení (můžete tedy vycházet z bloku Eng_1DO). Navíc však přibude analogový výstup nastavovaný PID regulátorem. V bloku Heater máte veškeré použití PID regulátoru předvyplněno, seznamte se se syntaxí.

10.3.6. Test funkčnosti modulu

Po implementaci kódu funkčního bloku Engine_1DO, je nutné otestovat funkčnost. V první fázi je vhodné otevřít si datový blok, monitorovat stavy a nastavovat příkazy. Vepsáním nové hodnoty do sloupce **Prepared value** a stiskem ikony  dojde k jejímu zapsání

TwinCAT_Project1.Untitled1.Main_Batch.FB_Tank_1.MixerCm

Expression	Type	Value	Prepared value
iRunning	BOOL	FALSE	
iError	BOOL	FALSE	
iSecImp	BOOL	TRUE	
oRun	BOOL	FALSE	
ioEngStruct	REFERENCE TO...		
Cmd	TEngineCmd		
SetRT	DINT	-1	
AutCtrl	BOOL	FALSE	
ManCtrl	BOOL	FALSE	TRUE
TurnOnMan	BOOL	FALSE	
TurnOffMan	BOOL	FALSE	
Req2RunAutCtrl	BOOL	FALSE	
ResetSUF	BOOL	FALSE	
State	TEngineState		
RT	DINT	315	
Run	BOOL	FALSE	
RunOut	BOOL	FALSE	
Err	BOOL	FALSE	
StartUpFail	BOOL	FALSE	
AutCtrl	BOOL	TRUE	
ManCtrl	BOOL	FALSE	
MayRun	BOOL	FALSE	
MayAutoRun	BOOL	TRUE	
GenErr	BOOL	FALSE	
StateInt	INT	1	
StartupFailTime	INT	0	
Time2Start	INT	0	
secImpEdge	BOOL	FALSE	
tempStartupFail	BOOL	FALSE	

Obrázek 46 Testování modulů zařízení

Během testování samozřejmě musíte mít spuštěn simulátor technologie. Dejte pozor, jestli komunikace se simulátorem funguje (zjistíte prozkoumáním hodnot stavů Modbus TCP komunikace).

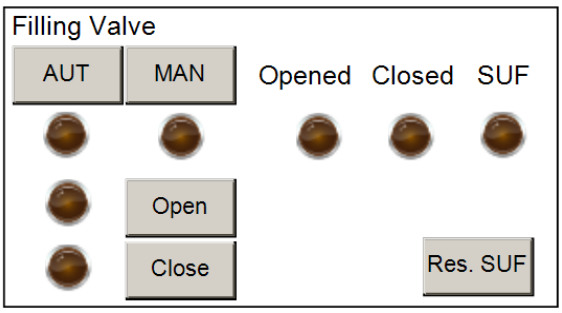
10.3.7. Připojení vizualizace k modulu

Nejsnáze lze zkontrolovat funkčnost modulu ve chvíli, kdy je připojen k vizualizaci. Tu si však napřed musíme vytvořit.

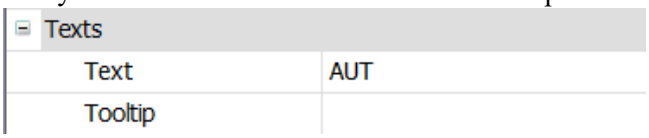
1. Klepneme pravým tlačítkem na záložku **VISUs** PLC projektu a zvolíme Add - Visualisation. Pokud jsme právě vytvořili naši první obrazovku vizualizace, vloží se projektu také položka

Visualisation Manager, pomocí které je možné nastavovat parametry vizualizace (např. startovací obrazovku).

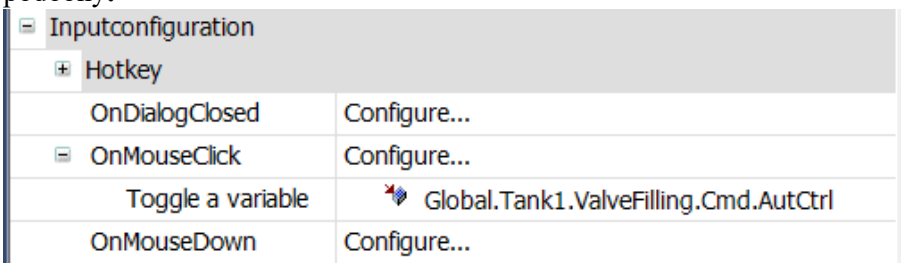
2. Na pracovní plochu naskládáme potřebné vizualizační prvky výběrem z toolboxu. Můžeme použít např. Rectangle - rámeček, Label - statický text, Lamp - kontrolku, Button - tlačítko. Návrh vizualizace pro jeden modul řízení může vypadat např. takto:



3. Texty tlačítek můžeme změnit v panelu **Properties** ve skupině **Texts:**



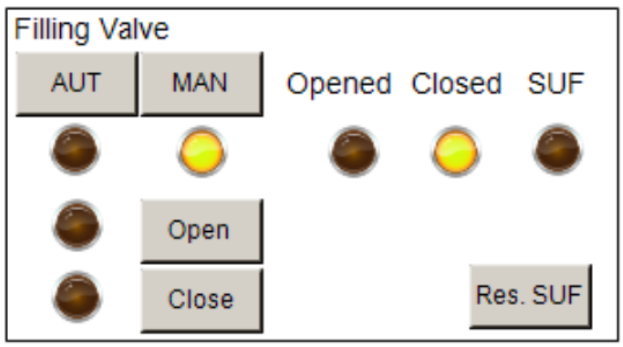
4. Chování tlačítek pak ve skupině **Inputconfiguration**, kde pro událost OnMouseClick zvolíme chování **Toggle Variable** a vybereme příslušnou proměnnou ze seznamu. Výsledek bude podobný**:**



5. Proměnnou navázanou na indikátor pak změníme pomocí vlastnosti **Variable**:



6. Po spuštění programu pak můžeme přejít na stránku s vizualizací a můžeme nyní ovládat chování řídicího modulu napouštěcího ventilu pomocí aktivních prvků:



7. Můžete vizualizovat také průběh komunikace s technologií přes Modbus TCP. Vyhnete se tím během ladění nepříjemnostem…

Kvalita vizualizace nebude předmětem hodnocení, je tedy zcela na Vás, jak ji pojmete. Pomocí vizualizace však budete při obhajobě prezentovat funkčnost Vašeho programu, tedy pokuste se vizualizaci udělat tak, aby toto bylo možné a nepřipravili jste se zbytečně o body.

10.3.8. Implementace Fáze

K tomuto bodu nepřistupujte, pokud nemáte dokončeny, otestovány a připojeny k vizualizaci všechny moduly zařízení.

Technologický proces probíhající v tanku se rozdělí do čtyř fází:

- Fáze napouštění
- Fáze vypouštění
- Fáze míchání
- Fáze ohřevu

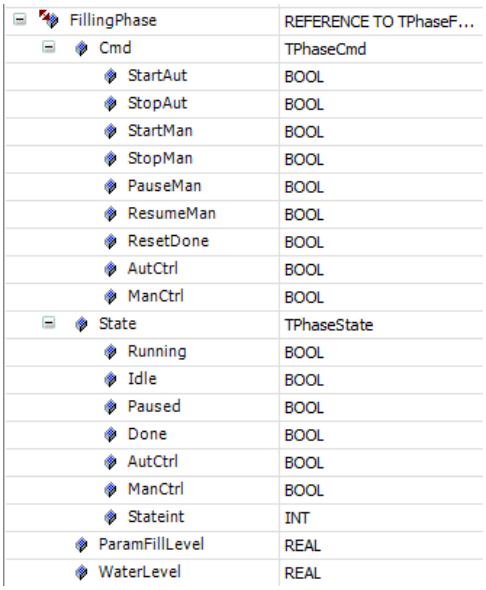
Stejně jako pro moduly zařízení, tak i pro fáze jsou již přichystány DUT a funkční bloky, které je potřeba doprogramovat.

Je nutné si uvědomit hierarchii řízení řízeného zařízení - řízené zařízení je možno, v našem případě, ovládat na nejnižší úrovni pomocí modulu zařízení - pokud je zařízení v manuálním režimu, je možné jej ovládat pomocí povelů zapnout a vypnout, přepne-li se modul zařízení do automatického režimu, znamená to, že řízené zařízení je možné ovládat nadřazeným řízením, v našem případě se jedná o fázi. Fáze v tomto režimu získává pozici subfunkce, která má své povely a stavy. Každou fázi (jak již bylo zmíněno výše) lze také přepínat mezi automatickým a manuálním režimem. Tzn., že pokud je fáze v manuálním režimu, je možné ji zapnout nebo vypnout z operátorského panelu pomocí příkazů zapnout nebo vypnout fázi. Fáze může obsahovat alespoň jeden modul zařízení, ale pro složitější fáze (ne v této demonstrační úloze) může obsahovat i několik modulů zařízení.

Rozdíl mezi spouštěním modulu zařízení a fáze vychází z principu požadavků na fázi - každá fáze má definované ukončení - pokud máme míchadlo, může se např. jednat o vypršení doby chodu míchadla, pro zahřívání dosažení požadovaného setpointu teploty apod. Naopak při manuálním ovládání modulu zařízení je vypnutí vždy na operátorovi či obsluze, která je v tomto režimu za chování systému zodpovědná.

Stejně jako u modulů zařízení je nutné podívat se na vstupy/výstupy jednotlivých funkčních bloků a zjistit význam jednotlivých proměnných v předdefinovaném DUT.

DUT pro fáze není možné měnit ani nijak upravovat - jakýkoli zásah způsobí nekompatibilitu s MES systémem SkuMES.



FillingPhase	REFERENCE TO TPhaseF...
Cmd	TPhaseCmd
StartAut	BOOL
StopAut	BOOL
StartMan	BOOL
StopMan	BOOL
PauseMan	BOOL
ResumeMan	BOOL
ResetDone	BOOL
AutCtrl	BOOL
ManCtrl	BOOL
State	TPhaseState
Running	BOOL
Idle	BOOL
Paused	BOOL
Done	BOOL
AutCtrl	BOOL
ManCtrl	BOOL
Stateint	INT
ParamFillLevel	REAL
WaterLevel	REAL

Obrázek 50 DUT fáze napouštění

Na Obr. 50 je definována struktura pro fázi napouštění. Význam jednotlivých proměnných je následující:

Příkazy (Cmd):

- ***StartAut*** – pokud je fáze v automatickém režimu MES systém pomocí této proměnné odstartuje fázi (přepne ji do stavu Running).
- ***StopAut*** – v případě poruchy systému, nebo příkazu procesního inženýra na zastavení fáze (dávky) se tímto signálem fáze ukončí (přepne se do stavu Idle).
- ***Start/StopMan*** - stejně jako u předchozích dvou signálu, ale funguje pouze, pokud je fáze v režimu manuálním.
- Nádstavba proti automatickému režimu je, že v manuálním režimu se objevují povely PauseMan a ResumeMan, z důvodu testování fází pro operátora/procesního inženýra. V důsledku to znamená např., že se spustí fáze míchání, po několika vteřinách se proces míchání pozastaví (přepne do stavu Paused), míchadlo je vypnuté, je zkontrolován obsah v tanku a poté se stiskne ResumeMan a fáze pokračuje dále po dobu zbývajícího času.
- ManCtrl/AutCtrl - přepínání mezi režimy manuál a automat (stejně jako u modulu zařízení).
- Reset Done - signál, který používá MES systém pro vyresetování stavu Done.

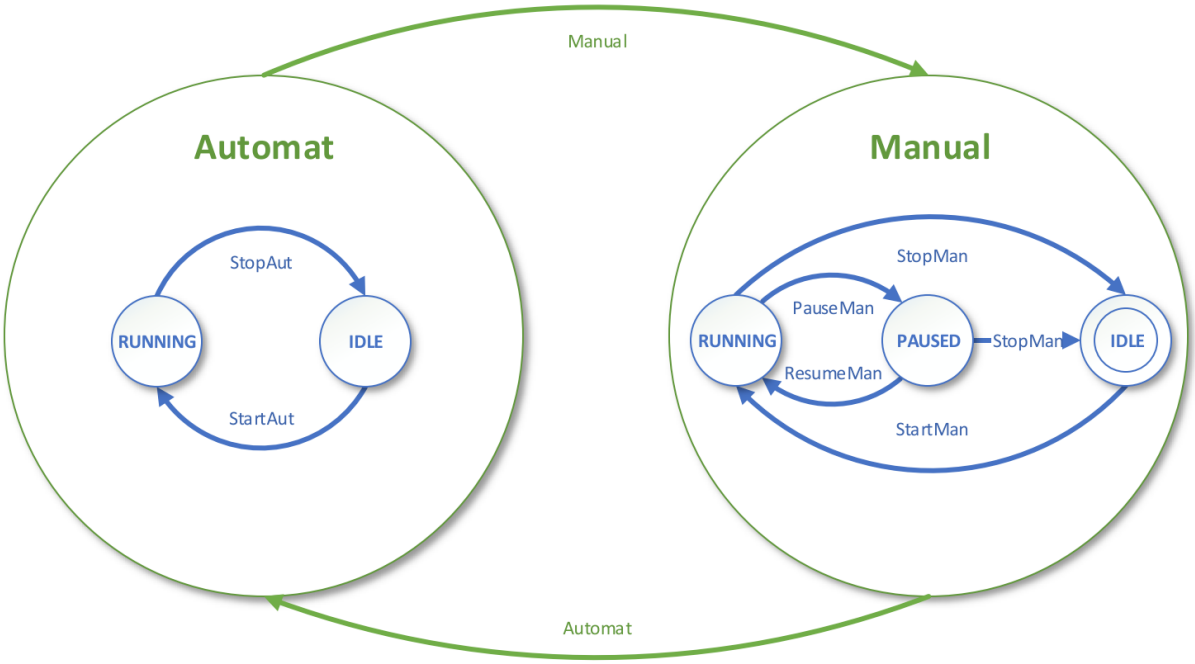
Stavy (State):

- Running/Idle/Paused - fáze je v daném režimu.
- Done - signál který se vygeneruje po dokončení fáze. Signalizace pro MES o tom, že byla fáze dokončena.
- AutCtrl/ManCtrl - signalizace o tom v jakém režimu je fáze.
- StateInt - číselná hodnota stavu, pro jednodušší zobrazení ve vizualizaci.

Parametry (Param) :

- FillLevel - slouží pro definování ukončovací podmínky fáze. Napouštění probíhá, dokud hladina nedosáhne požadovaného setpointu.

Fáze také musí znát aktuální hodnotu hladiny, která se fázi vkládá pomocí proměnné WaterLevel – tato hodnota je porovnávána s FillLevel a na základě splnění podmínky se fáze ukončí.



Obrázek 51 Stavy a povely fáze

Nyní přichází čas na programování. Dejte se do něj.

Region	Popis
Aut/Man	V rámci tohoto regionu se na základě příkazů AutCtrl a ManCtrl nastaví příslušný stav fáze napouštění. Dále pak zde ošetříme případ, kdy není aktivní ani jeden stavový příznak (AutCtrl či ManCtrl). V takovém případě nastavíme manuální režim. Všimněte si, že příkaz pro nastavení automatického režimu je Cmd.AutCtrl, kdežto příznak (stav) aktivity je State.AutCtrl.
ManCtrl	Pokud je fáze v manuálním režimu, zpracovávají se v tomto regionu příkazy pro změnu stavu – StartMan, StopMan, PauseMan, ResumeMan. Na základě těchto příkazů se přepíná stav automatu fáze mezi stavy Running, Idle, Paused. Tato sekce pouze definuje přechody automatu, nezabývá se činnostmi, které má fáze vykonávat v jednotlivých stavech.

	Pozn.: Při zpracování každého příkazy proveďte reset příznaku Done. Po zpracování konkrétního příkazu jej resetujte.
AutCtrl	Pokud je fáze v automatickém režimu, zpracovávají se v tomto regionu příkazy StartAut a StopAut. Příkazy opět realizují přechody stavového automatu fáze. Opět je po zpracování resetujte.
Done	Zde budeme realizovat 1) Testování, zda je splněna ukončovací podmínka fáze (v případě napouštění tedy, že je hladina vody vyšší nebo rovna žádané hladině). Pokud ano, nastavíme příznaky StopAut a StopMan, které zabezpečí korektní ukončení fáze. Dále pak nastavíme příkaz Done, který indikuje dokončení fáze nadřazenému systému. 2) Obsluhu příkazu ResetDone. Pokud bude nastaven, vyresetujeme příznak Done. Nezapomeneme opět resetovat i příkaz ResetDone.
States	V tomto regionu budeme realizovat vlastní činnost fáze. 1) Pokud fáze běží, nastavíme modulu zařízení příkaz Req2OpenAut (a pro pořádek mu vymažeme příkaz Req2CloseAut). 2) Pokud fáze neběží, provedeme opak.
„“	V poslední části, která není (ale může být) obalena regionem provedeme 1) Nastavení defaultního stavu (Idle) v případě, že není aktivní ani jeden ze stavových příznaků 2) Nastavení proměnné Stateint na hodnoty #Idle, #Running či #Pause podle aktuálního stavu. Tato proměnná existuje kvůli efektivnějšímu zpracování vizualizace.

10.3.9. Testujeme fázi napouštění

Stejně jako u modulů zařízení, tak i fází je třeba otestovat funkčnosti. Pro začátek je možné testovat z DB posíláním povelů a sledováním chování řízených zařízení.

Funkčnost fáze demonstrujte vyučujícímu.

10.3.10. Vytvoříme vizualizaci

Tagy potřebné na propojení PLC s HMI jsou opět předdefinované, je však třeba přidat funkcionalitu tlačítkám, zobrazovacím okénkům apod.

Funkčnost vizualizace demonstrujte vyučujícímu.

10.3.11. Doprogramujeme ostatní fáze

Fáze pro demonstrační úlohu jsou 4.

Pro fázi zahřívání je zajímavý region Done:

Done	V tomto regionu je nutno testovat jako ukončovací podmínku nejen dosažení žádané hodnoty, ale také dle zadání, zda hodnota za posledních 10 sekund nevystoupila z intervalu ±1 °C. K tomu je vhodné využít sekundový vstup, který je přiveden na vstup každé fáze (i každého modulu zařízení). V případě, kdy je fáze spuštěná, je nutné testovat, zda se skutečná hodnota pohybuje v žádaném rozmezí. V případě, že ano a současně je aktivní sekundový vstup (vždy pouze 1x za sekundu), přičte se do interní proměnné fTime hodnota 1. Pokud tato proměnná nabyde hodnoty >= 10, je možné fázi ukončit. Pokud ale v průběhu počítání žádaná hodnota opustí definovaný interval, je nutné fTime vynulovat. Stejně tak ji vynulujte vždy, pokud fáze neběží.

10.3.12. Testujeme

…

10.3.13. Vytvoříme vizualizaci

Bude doplněno, nicméně vizualizace fází může vypadat obdobně, jako vizualizace řídicích modulů, jen trochu jinými tlačítky a indikátory, odpovídající Cmd a State, případně analogovými hodnotami (teplota, výška hladiny, výkon topení).

10.4. Automatické spouštění fází stavovým automatem v PLC

Než se vše spojí s MES systémem, je možné zkusit si naprogramovat výrobní proces v PLC. V důsledku to znamená, že vytvoříte stavový automat ve strukturovaném textu, který bude jednotlivé fáze parametrizovat spouštět v definovaném pořadí.

```
CASE State OF
    1:  Global.Tank1.Filling.ParamFillLevel := 1.8;
        Global.Tank1.Filling.Cmd.ManCtrl := TRUE;
        Global.Tank1.Filling.Cmd.StartMan:= TRUE;
        IF Global.Tank1.Filling.State.Done = TRUE THEN
            State := 2;
        END_IF
    2: //...
END_CASE
```

Napustit, míchat + topit, vypustit. Pro pokročilejší – mělo by být možné definovat množství dávek a pak celý proces spustit a vyrobit předdefinovaný počet dávek.

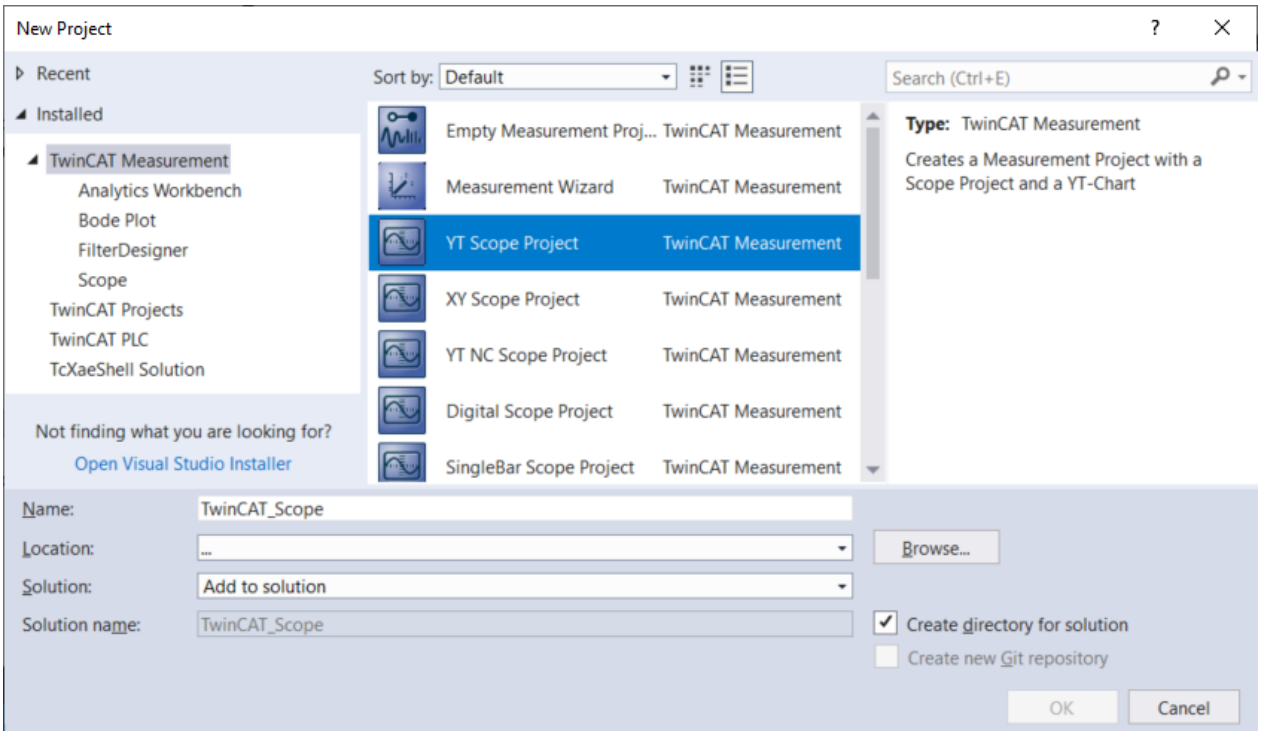
11. Optimalizace regulačního děje – návrh regulátoru pomocí nástroje MATLAB

Pro návrh regulátoru je třeba mít představu o dynamickém chování řízené soustavy. Pro tyto účely je třeba provést proces identifikace a následně syntézu (návrh) regulátoru. V následujícím testu budou popsány tyto procesy využitím nástrojů, které poskytuje prostředí MATLAB.

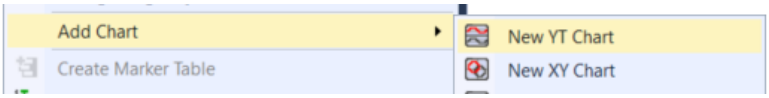
11.1. Tvorba dat pro MATLAB

Data pro MATLAB můžeme získat několika způsoby. Jedním z nich je např. využití knihovního funkčního bloku **FB_CTRL_LOG_DATA**, který umí zapisovat do CSV datového souboru. My si ale ukážeme jinou cestu, která se více hodí pro ladění a zahrnuje vzorkování dat, zpracování a vizualizaci přímo v systému TwinCAT.

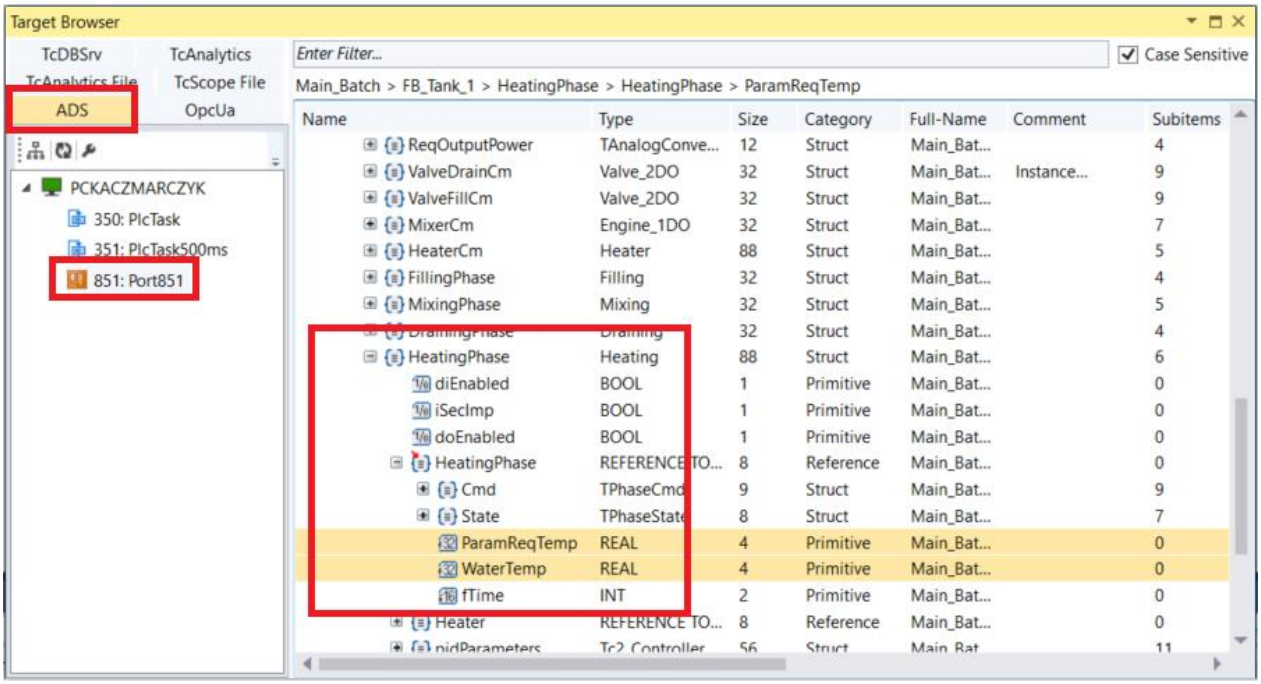
Vytvoříme si ladicí projekt, prostřednictvím kterého budeme schopni vizualizovat data. Přidejme si tedy do našeho Solution projekt **YT Scope Project** skupiny **TwinCAT Measurement – Scope**. V případě, že skupinu TwinCAT Measurement nemáme instalovánu, je nutno doinstalovat balíček **TF3300 | TwinCAT 3 Scope Server** ze stránek BECKHOFF.



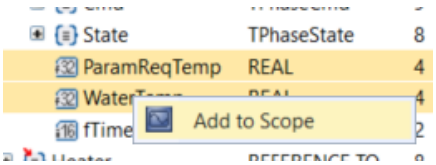
Pole **Location** ponechejte stejné, jako je umístění vašeho projektu **Tanky**. Po vytvoření projektu dále klikem pravým tlačítkem myší na položku projektu v **Solution Explorer** zadejte **Add Chart – New YT Chart**. Tím dojde k vytvoření nového grafu.



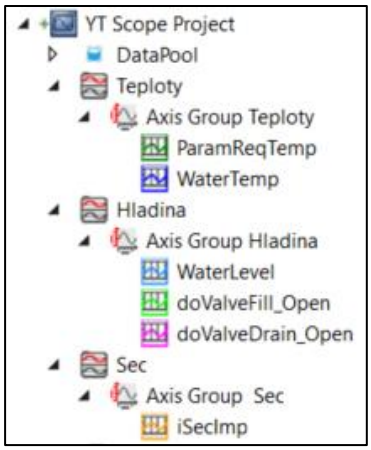
Nyní klikněte pravým tlačítkem opět na položku **YT Chart** ve stromu projektu a vyberte **Add – New Axes** a následně na položku **Axis Group** pravým tlačítkem a zadejte **Target Browser**. Ujistěte se, že máte vybrán paměťový prostor ADS, dále vyberte v lokálním PC (Soft PLC) položku Port851 a pak v hlavní části okna rozbalte strukturu dle obrázku níže.



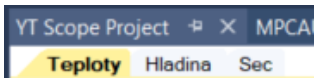
Pomocí Ctrl + klik myší vyberte dvě hodnoty – žádanou teplotu vody a skutečnou teplotu vody a přes pravé tlačítko myši vyberte **Add to Scope**. Na dotaz, zda se má vytvořit separátní kanál pro každý ze symbol dejte **Ne**. Tato odpověď zabezpečí vložení obou proměnných do jednoho souřadného systému.



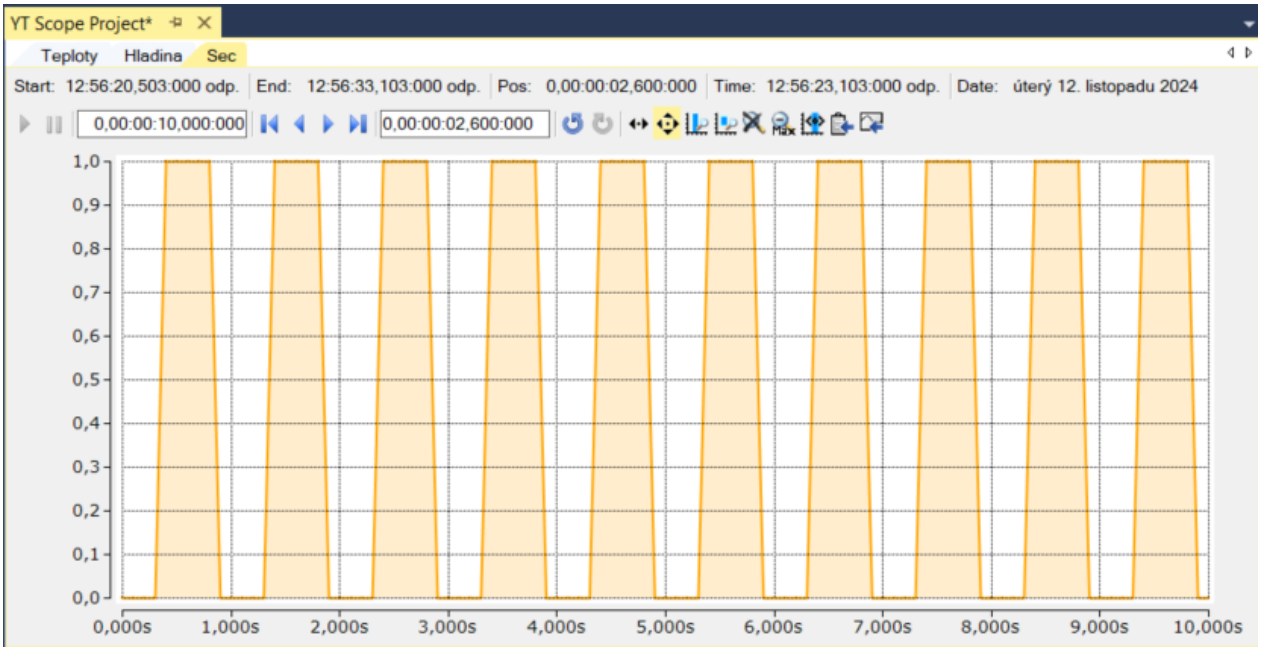
Do projektu můžeme vložit více grafů. Na obrázku níže vidíme vložený ještě graf pro zobrazení výšky hladiny a stavu napouštěcího a vypouštěcího ventilu a dále graf, ve kterém bude vizualizována funkčnost sekundového vstupu – to jen pro kontrolu funkce komunikace.



Po dokončení aktivujeme konfiguraci a spustíme program v SoftPLC. Nyní (pokud není) otevřeme kartu YT Scope Project dvojklikem na některý z definovaných grafů. Vidíme, že v této kartě je možno přepínat mezi zobrazením jednotlivých grafů.



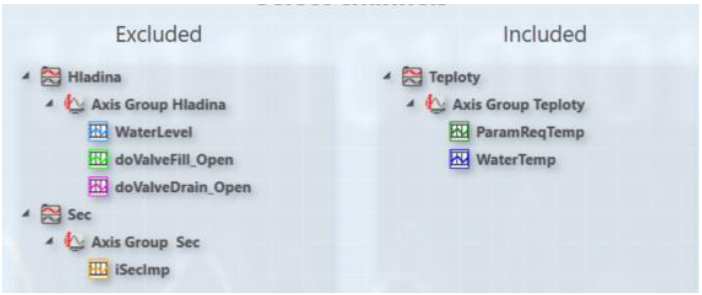
Pro start záznamu stiskneme ikonu **Start Record (F5)** z panelu  . Po nachytání dostatečného množství dat můžeme stiskem **Stop Record (F6)** záznam opět zastavit (první dvě ikony na následujícím obrázku).



Pro zpracování v programu Matlab potřebujeme data exportovat do souboru .CSV. Z hlavní nabídky vybereme volbu **Scope – Export** a postupujeme po krocích:

- Vybereme volbu **Character Separated Values**

- Vybereme pouze teploty



- Ponecháme doporučenou časovou periodu (uloží se celý záznam)
- Vybereme parametry pro export
  - Cull Timestamp
- Nastavíme CSV parametry
  - CSV-Separator: Semicolon
  - Decimal mark: Point
  - Header configuration: None
- Vybereme název umístění souboru
  - Název: sys_data.csv

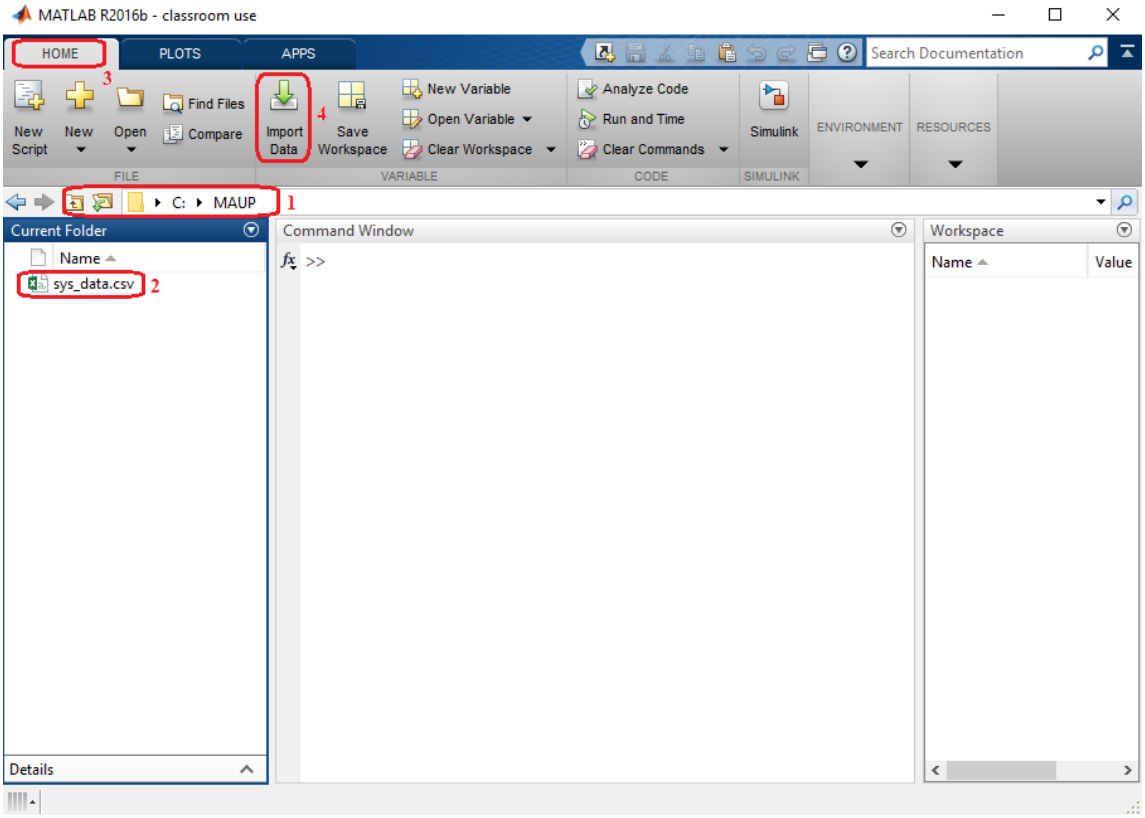
Pokud si následně soubor zobrazíme, uvidíme následující:

```
133758861805030000;0;133758861805030000;0
133758861806030000;0;133758861806030000;0
133758861807030000;0;133758861807030000;0
133758861808030000;0;133758861808030000;0
133758861809030000;0;133758861809030000;0
133758861810030000;0;133758861810030000;0
133758861811030000;0;133758861811030000;0
```

První a třetí hodnota udává časovou značku měření ve formátu **8-digit LDAP/FILETIME** (pro detail viz https://www.epochconverter.com/ldap). Druhá a čtvrtá hodnota pak udávají žádanou, resp. skutečnou hodnotu. Tyto budeme potřebovat pro identifikaci.

11.1.1. Načtení dat

V předchozím textu byl popsán proces získání (měření) vstupně/výstupních dat z PLC reprezentujících vstupní a výstupní signály z řízené soustavy ve formě změřené přechodové charakteristiky. Tato data byla uložena do souboru sys_data.csv. Naměřené hodnoty z tohoto souboru je nejprve nutné importovat do prostředí MATLAB, viz následující obrázek.

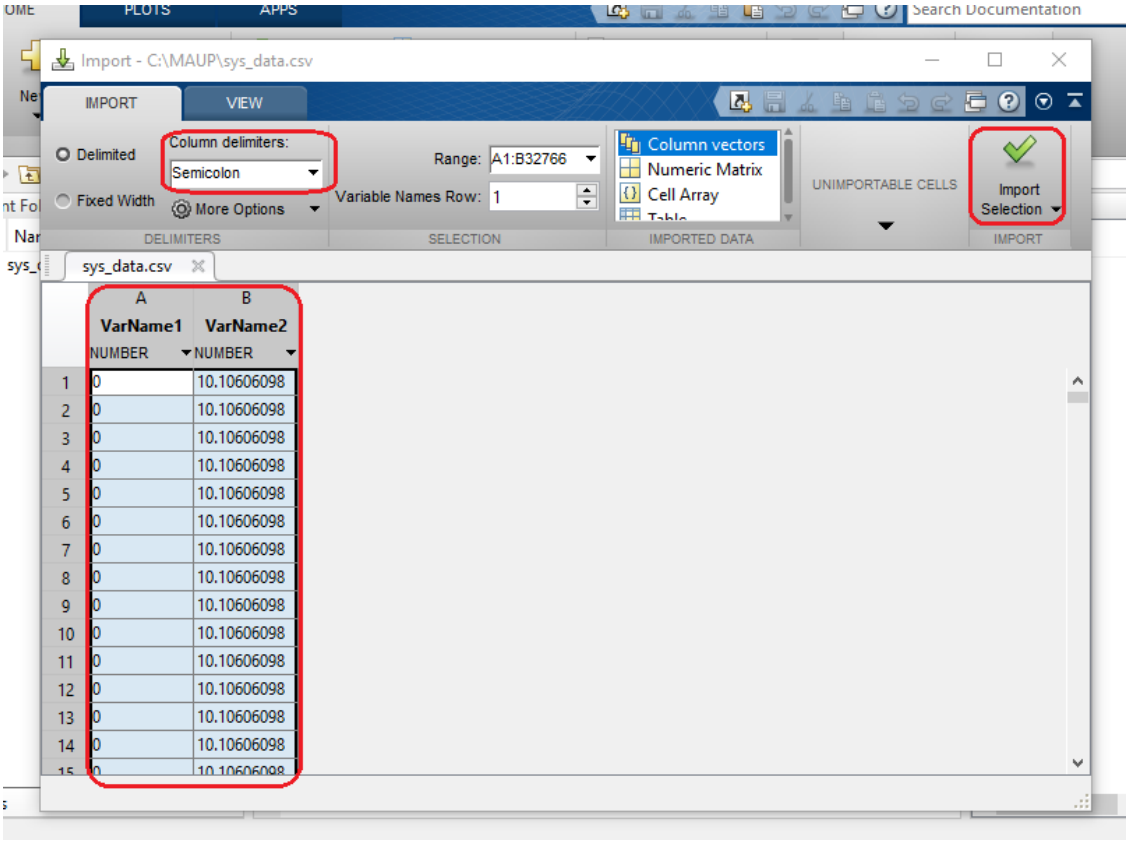


1. Nastavit pracovní adresář (1) – zvolte vlastní, případně dle pokynů vyučujícího.
2. Zkontrolovat, zda je v pracovním adresáři přítomen soubor sys_data.csv (2)
3. V záložce ***Home*** (3) kliknout na ***Import Data*** (4) a v dialogovém okně vybrat *.csv* soubor.

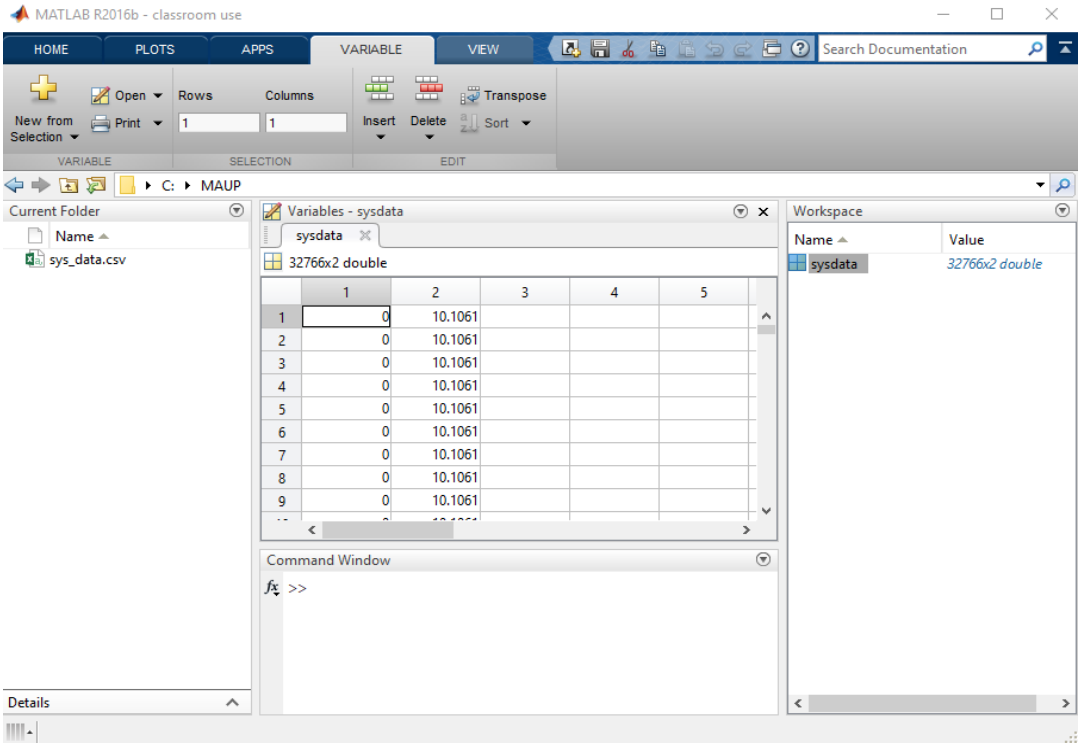
Otevře se nové okno, viz následující obrázek, a zde zkontrolujeme, zda jsou data správně rozparsována. Výsledek by měl vypadat jako na následujícím obrázku, tj. jsou k dispozici **2 oddělené sloupce** se vstupním a výstupním signálem. Pokud se sloupce nezobrazují správně, je třeba změnit typ oddělovače v menu ***Column delimiters***.

Pozn.: Pokud jsou v původním souboru data s desetinnou čárkou, je třeba změnit na desetinnou tečku!

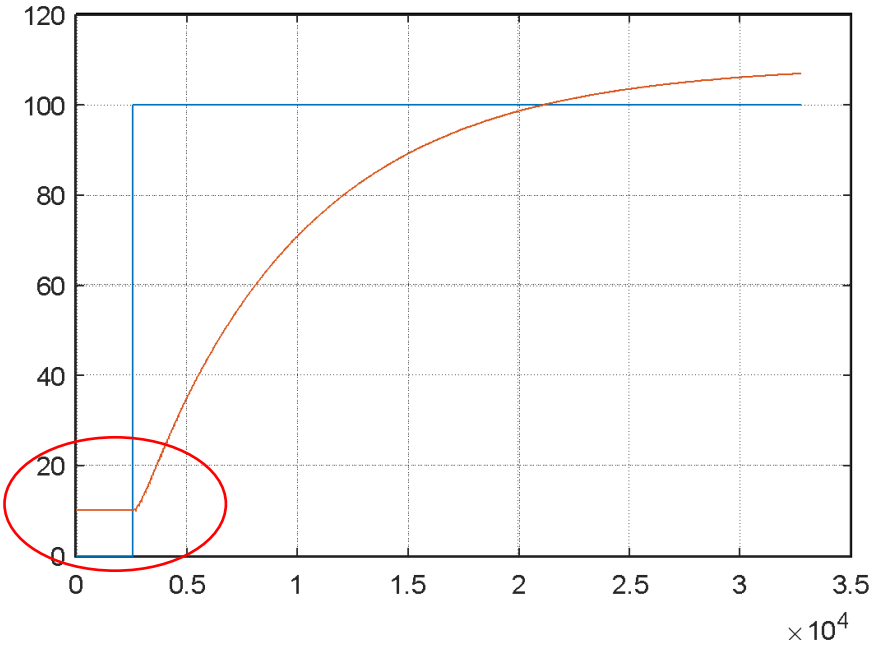
4. Poté je třeba vybrat datový typ, do jakého se budou data importovat. Vhodná je např. matice, tj. ***Numeric Matrix***.
5. Nakonec kliknout na Import selection.



Po chvíli by se měla zobrazit ve ***Workspace*** matice dat obsahující importovaná data z *.csv* souboru. Viz následující obrázek.



Zde je vhodné si data zobrazit v grafu (příkaz ***plot***), aby bylo vidět, s čím budeme dále pracovat.



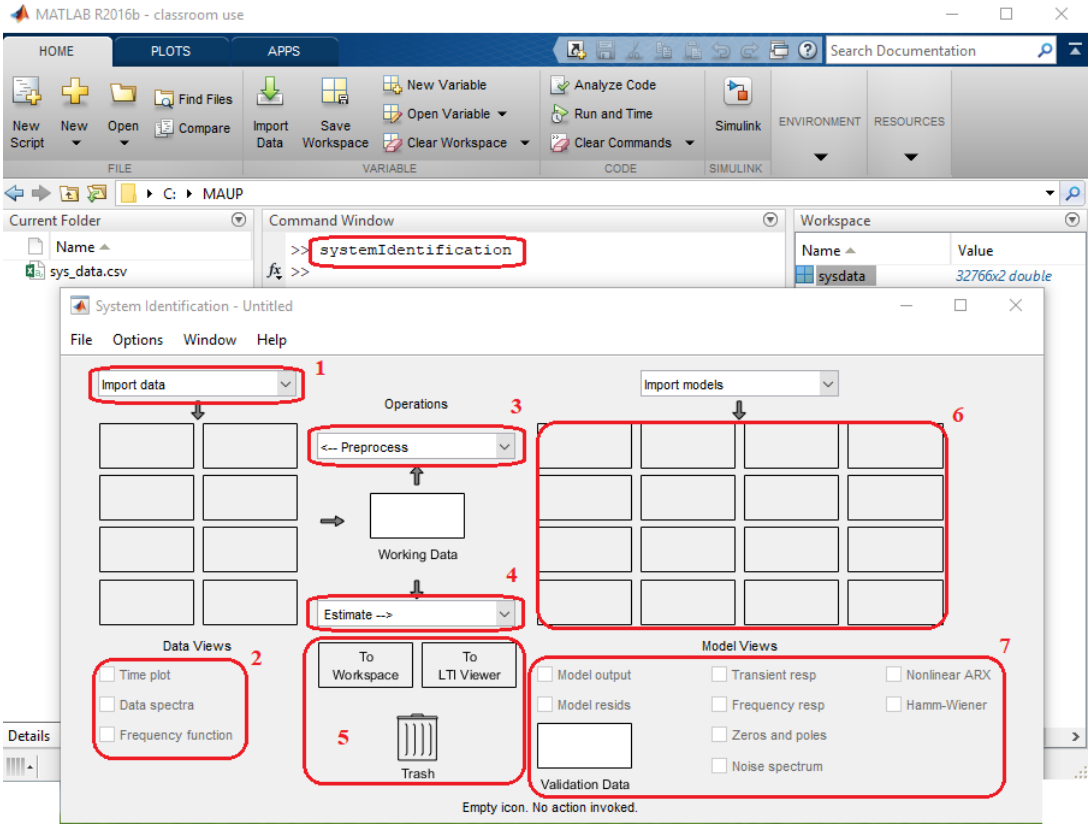
Uvedená data reprezentují přechodovou charakteristiku dynamického systému. Z této charakteristiky lze zjistit (stanovit) několik informací, a sice:

- typ soustavy: statická/astatická,
- zesílení soustavy,
- přibližně určit řád systému,
- počáteční podmínky.

Chystáme se dále identifikovat přenos systému, který je definovaný pro **nulové počáteční podmínky**. Je tedy třeba zajistit to, aby oba průběhy (vstupní i výstupní signál) splňovaly podmínku $f(0) = 0$.

11.1.2. Identifikace

Pro identifikaci parametrů soustavy je možné využít nástroj ***System Identification Toolbox***. Ten lze spustit např. zápisem příkazu ***systemIdentification*** (nebo ***ident***) v ***Command Window***. Otevře se následující okno.

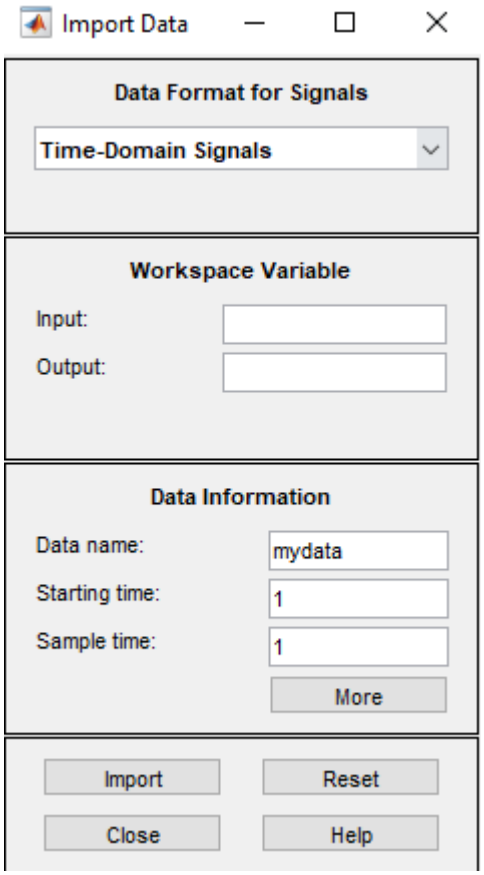


Popis sekcí v okně System Identification:

1. **Import data** – zde je možné našíst data v časové či frekvenční doméně, apod.
2. **Data Views** – zobrazení importovaných dat, případně zobrazení spektra, apod.
3. **Preprocess** – zde je možné provést preprocesing vstupních dat, tj. např. filtrovat apod.
4. **Estimate** – vlastní výběr způsobu identifikace
5. **Okno pro práci s identifikovaným modelem** – zde je možné výsledný model uložit do Workspace nebo jej smazat. Provádí se přetažením modelu ze sekce 6.
6. **Identifikované (resp. importované) modely –** zde se zobrazí výsledek identifikace. Pokud již máme např. uložený model ve Workspace, lze jej zde importovat
7. **Model Views –** zobrazení charakteristik identifikovaného modelu

Postup identifikace dat je v našem případě následující:

1. Rozbalit nabídku ***Import data*** a vybrat ***Time domain data…***
2. Zobrazí se vyskakovací okno, kde je třeba vybrat vstupní data a uvést další parametry, viz následující obrázek.



3. Nyní zadáme vstupní (***Input***)a výstupní (***Output***) data, pojmenovat datový soubor (***Data name***) a vybrat začátek (***Starting time***) a vzorkovací periodu (***Sample time***). V našem případě to bude:

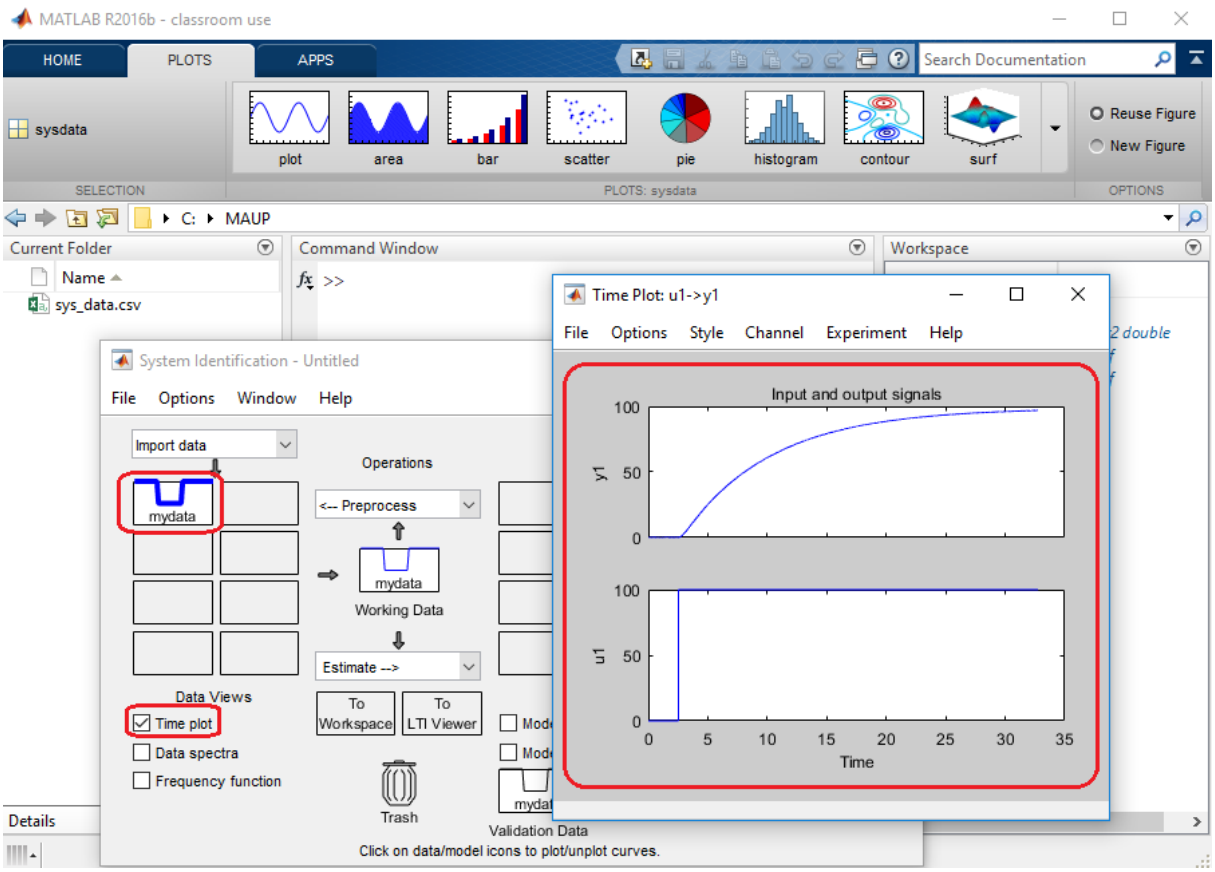
Input: sysdata(:,1) všechny řádky, 1. sloupec

Output: sysdata(:,2) všechny řádky, 2. sloupec

Starting time: 0

Sample time: 0.1 (pokud vzorkovací perioda, s jakou jste ukládali data v PLC byla 100 ms)

4. Kliknout na ***Import***
5. V sekci Import data se zobrazí importovaná data, která lze zobrazit zaškrtnutím ***Time plot***

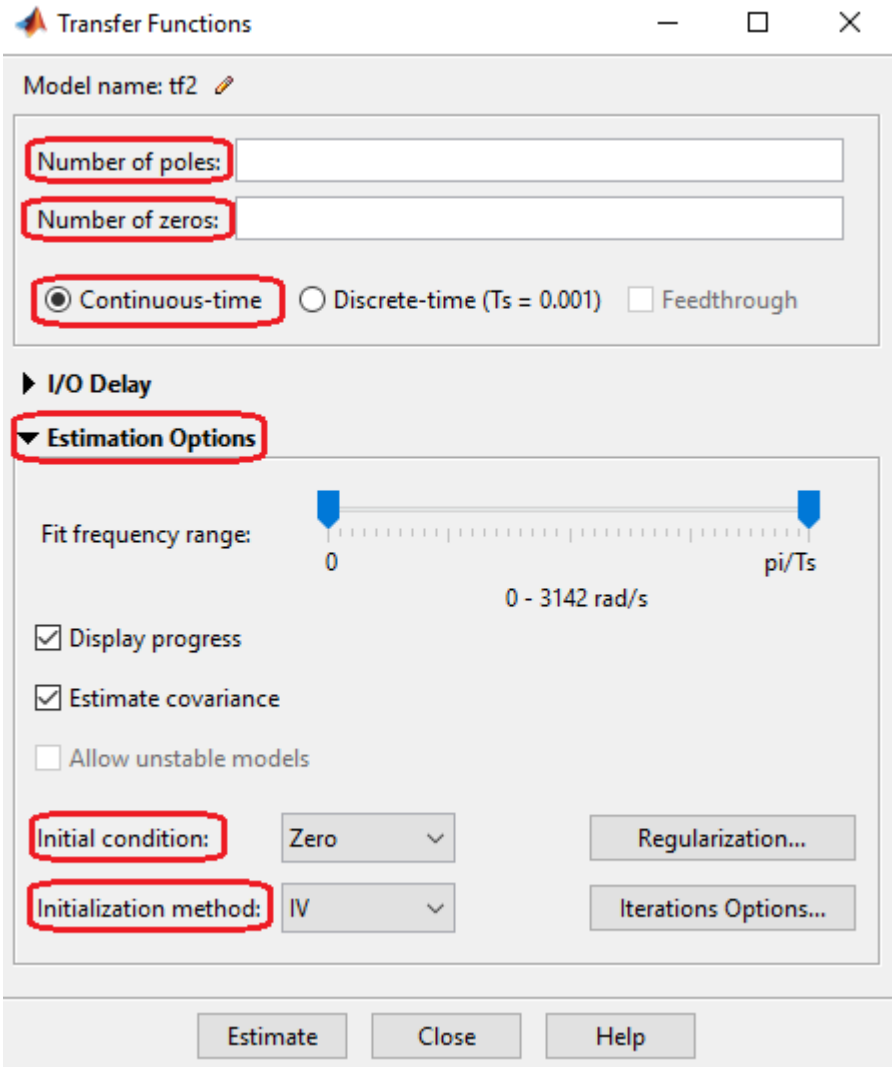


6. Vstupní data nejsou výrazně zašuměná a jsou tak připravena přímo pro identifikaci, viz nabídka ***Estimate***. Zde máme na výběr několik možností. Nejjednodušší způsob je např. ***Transfer Function Models***.

7. Otevře se následující okno, kde je třeba nastavit:

   - **počet pólů** (Number of poles)
   - **počet nul** (Number of zeros)
   - **spojitý** (Continuous-time) **model**
   - v záložce Estimation Options pak **počáteční podmínky** (Initial condition) na nula (Zero) a **inicializační metodu** můžeme nechat IV (Instrument Variable)

Počet pólů a nul zvolte s ohledem na tvar naměřené přechodové charakteristiky.

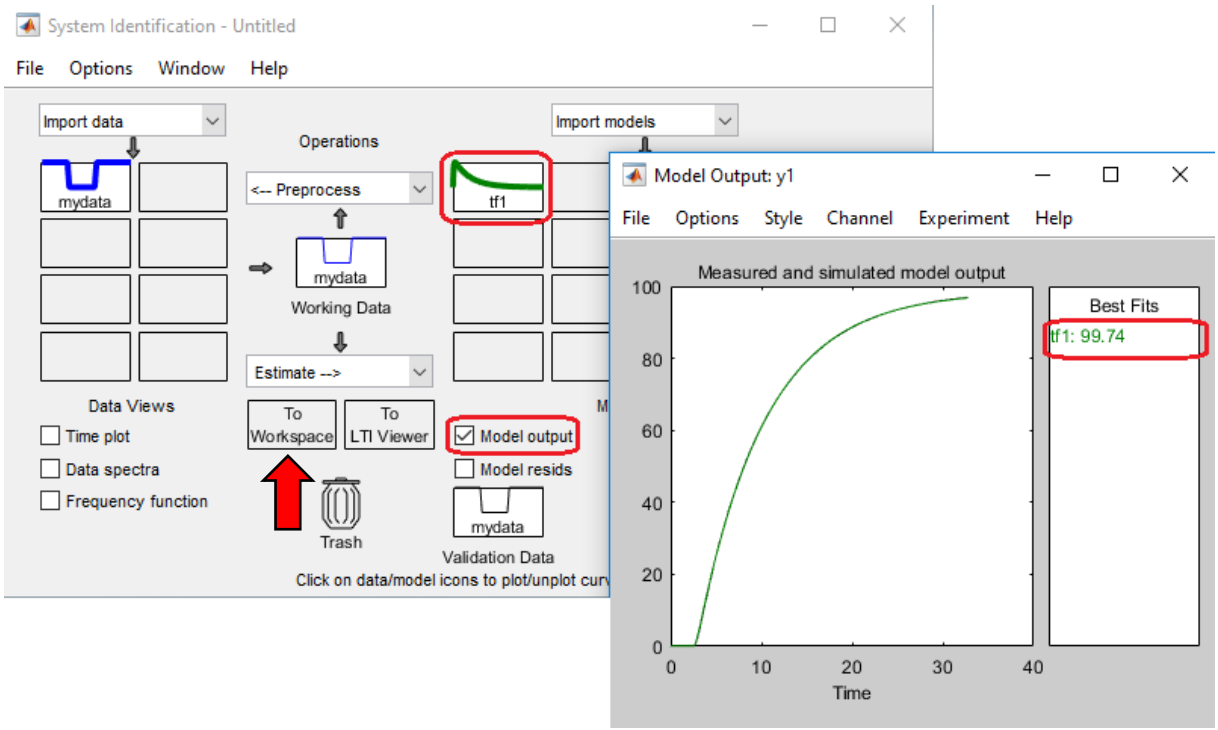


8. Poté stačí kliknout na ***Estimate***.

9. Po chvilce se zobrazí výsledek (identifikovaný model) a můžeme si ověřit úspěšnost identifikace např. kliknutím na ***Model output*** (v tomto případě je přesnost identifikace vyjádřená parametrem Best fit na hodnotě 99.74 %, což je velice dobrý výsledek), viz následující obrázek.

10. Nyní už můžeme importovat model do Workspace jeho přetažením do okna ***To Workspace***.

11. Ve Workspace už lze s modelem pracovat jako s klasickou přenosovou funkcí, tj. lze aplikovat standardní příkazy (pzmap, step, impulse, bode, nyquist, apod.)



11.1.3. Návrh regulátoru

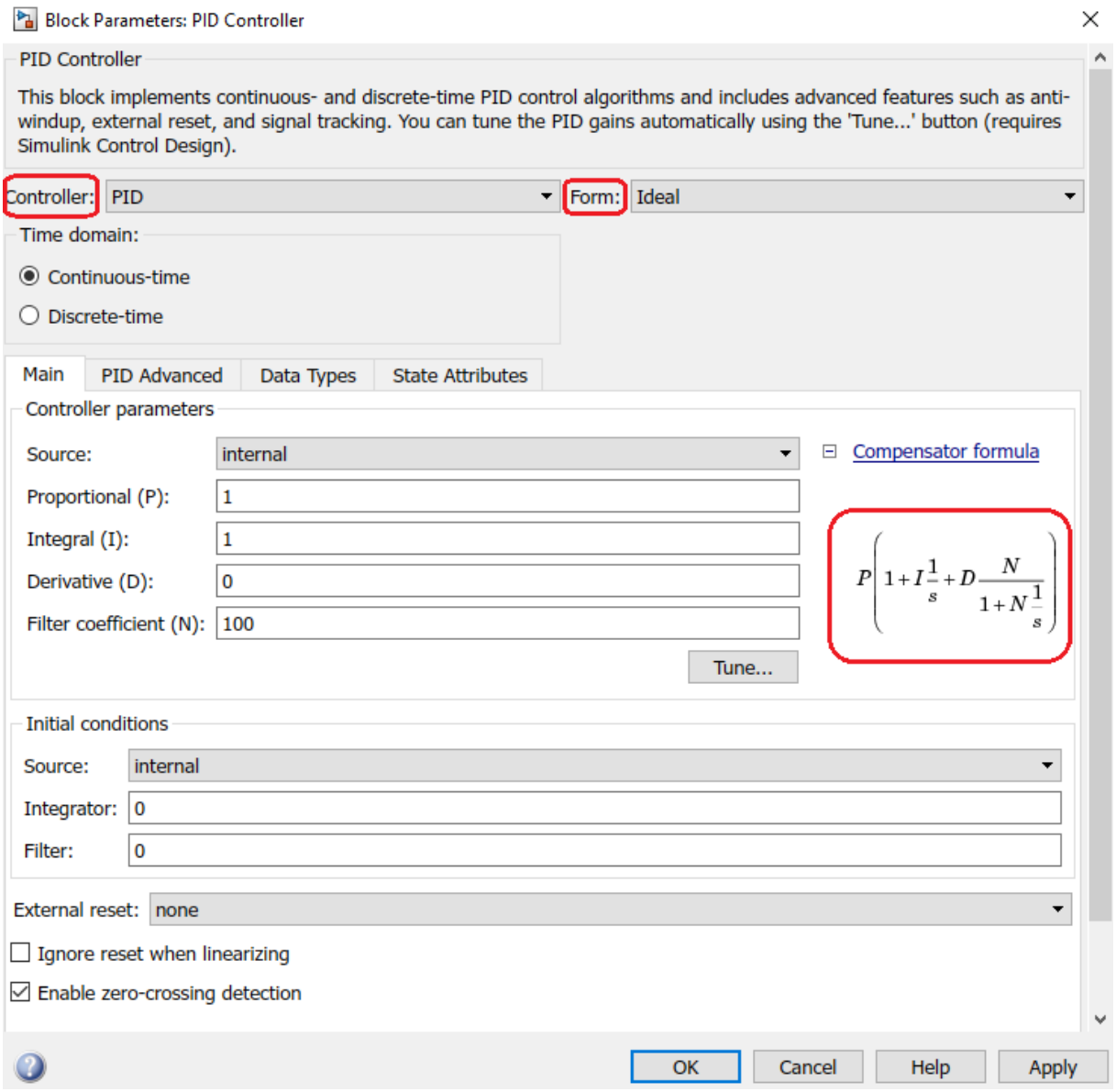
Pro návrh regulátoru je možné využít nástroje ***PID tune***. Ten lze spustit buď přímo z Command Window příkazem ***pidtool*** (zde jsou však omezené možnosti) nebo pracovat s tímto nástrojem v Simulinku.

Zde si v prvním kroku vytvoříme standardní regulační smyčku, kde jako soustavu vložíme identifikovaný model a jako regulátor blok ***PID controller*** z nabídky ***Continuous***.

Neměli bychom zapomínat na omezení, které v obvodu máme, a sice omezený akční zásah. Vložíme tedy ještě blok ***Saturation*** z nabídky ***Discontinuities*** a nastavíme horní a dolní mez. Výsledné schéma by mělo vypadat následovně.

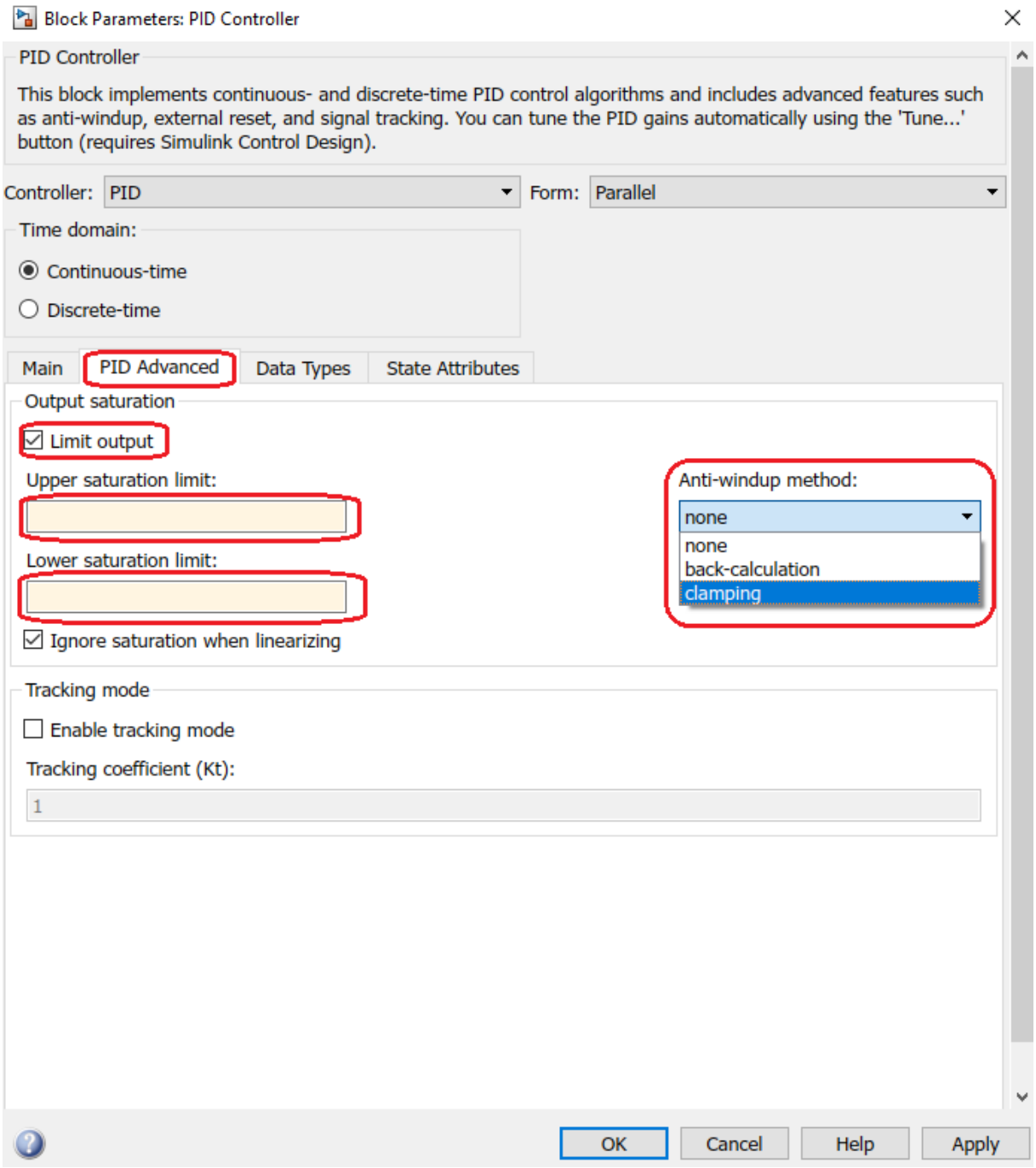


Nastavení regulátoru provedeme z nabídky PID tool, kterou otevřeme dvojklikem na PID Controller. Zde vybereme vhodný typ regulátoru (P, PI, PD, PID, apod.) a zvolíme formu zápisu – ***Ideal*** nebo ***Parallel***. Níže se nám tyto tvary zobrazí.



Vzhledem k tomu, že PID regulátor implementovaný v PLC Simatic využívá tvar odpovídající možnosti ***Ideal***, zvolíme tento tvar. Ladění si ukážeme např. na **PI regulátoru**.

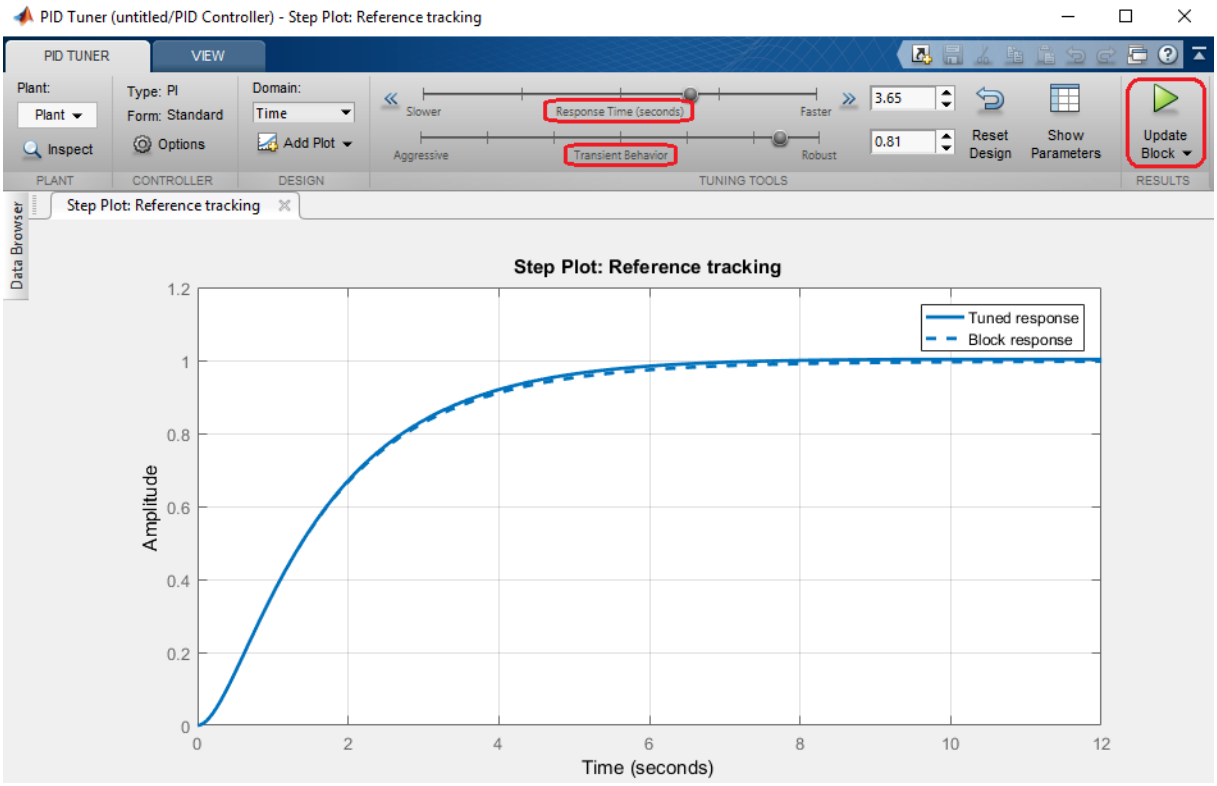
Výše bylo zmíněno omezení akčního zásahu. To je třeba zohlednit i při návrhu regulátoru a lze provést nastavením tohoto omezení (Limit output) v záložce ***PID Advanced***. Ve stejné záložce lze také nastavit metoda ***Anti-windup*** (např. ***Clamping***).



Vrátíme se do záložky ***Main*** a klikneme na tlačítko ***Tune***.

Otevře se okno, kde můžeme jednoduše pomocí posuvníků ***Response time*** a ***Transient Behavior*** měnit vlastnosti regulátoru. Výsledná odezva regulátoru se zobrazuje v grafu.

Až budeme s průběhem (odezvou) spokojeni, můžeme kliknout na tlačítko ***Update block***.



Následně můžeme toto okno zavřít a vrátit se do okna ***PID Parameters***. Zde už jsou aktuální hodnoty složek P a I podle provedeného „návrhu“.

Regulátor můžeme otestovat pomocí simulace v Simulinku a pokud je vše tak, jak má být, je možné výsledné parametry použít (**ve správném tvaru**) pro blok PID regulátoru v PLC Simatic.

12. Programování typu BATCH – systém pro řízení výroby

Zadání: zadefinujte stanici PLC, její adresu

1. Na stanici vytvořte jednotku tanku.
2. Pro jednotku vytvořte fáze jednotky, které použijete v receptuře.
3. Vytvořte testovací recepturu pro ověření fáze napouštění a vypouštění
4. Vytvořte recepturu pro řízení celé technologie.

12.1. Nastavení projektu PLC pro komunikaci OPC-UA

Pro spuštění OPC-UA serveru v PLC a možnost jeho konfigurace v konfiguračním PC je nutné nainstalovat následující balíčky:

- TF6100-OPC-UA-Server-V5.2.exe (nutno instalovat do PLC i do PC v případě použití SoftPLC)
- TF6100-OPC-UA-Client.exe (testovací aplikace)
- TF6100-OPC-UA-Configurator.exe (konfigurační aplikace)

Po nainstalování balíčků je nutné v aplikaci TwinCAT provést aktivaci licence stejně, jako jste aktivovali licenci pro Modbus TCP.

TF6100	TC3 OPC-UA	cpu lic...		expires ...	BDCC0070-42D5-49AE-ABF1-1D4...

V dalším kroku je nutné systému říci, které proměnné budou komunikovány prostřednictvím OPC-UA serveru. Toto se nastaví přímo v definici bloku globálních proměnných takto:

```
{attribute 'qualified_only'}
VAR_GLOBAL
    {attribute 'OPC.UA.DA' := '1'}
    {attribute 'OPC.UA.DA.StructuredType' := '1'}
    Tank1 : TTank;

    {attribute 'OPC.UA.DA' := '1'}
    test_opc : INT := 1;
```

První přidaný atribut ***qualified_only*** označuje, že k proměnným v následujícím datovém bloku bude možné přistupovat pouze prostřednictvím jejich plného názvu (např. tedy Global.Tank1, pokud se datový blok jmenuje Global).

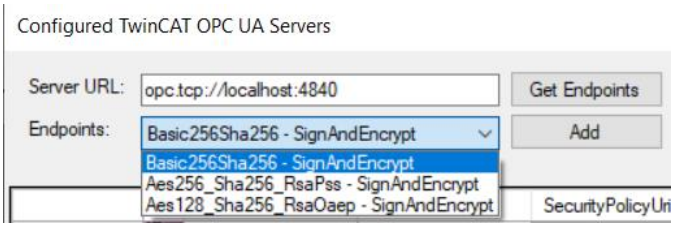
Další atribut ***OPC.UA.DA*** nastavený na hodnotu 1 označuje následující symbol (proměnnou) jako komunikovanou prostřednictvím OPC-UA (DA = Data Access). Atribut ***OPC.UA.DA.StructuredType*** pak označuje datový typ jako strukturovaný. Pro Tank1 je nutné jej použít, pro test_opc hodnotu typu INT zjevně nikoli.

Dále je potřebné vygenerovat v PLC projektu popisný OPC UA **TMC** soubor.

TBD: Kubo, zde se můžeš vypsat

Nyní máme projekt připraven pro spuštění s OPC-UA serverem. Provedeme aktivaci konfigurace a odstraníme případné chyby překladu. Dále provedeme spuštění v SoftPLC.

Dalším krokem je spuštění OPC UA Konfigurátoru a stisknutí tlačítka **Edit** na liště. Jako URL serveru zadáme adresu **opc.tcp://localhost:4840** a zmáčkneme tlačítko **Get Endpoints**. Ze seznamu endpointů vybereme první a stiskneme tlačítko **Add**.



Zavřeme okno přidání UA serveru. Následně na liště vybereme tento server z rozbalovacího seznamu a stiskneme tlačítko **Connect**. Měl by se objevit dialog pro zadání přístupových údajů. Jedná se o údaje, které budou **při inicializaci serveru** (kterou nyní procházíme) **nastaveny** (tedy nikoli o údaje, které nastavil před Vámi někdo jiný!). Zadejte tedy

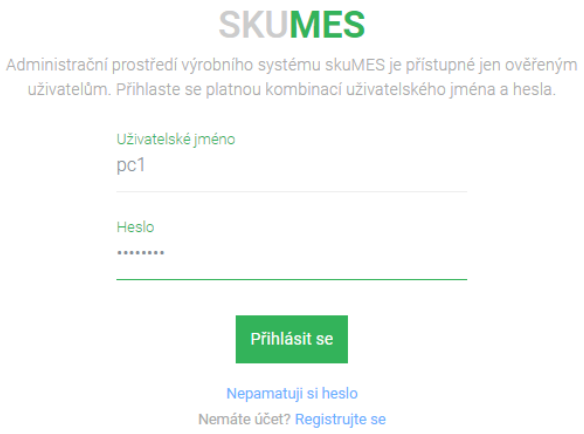
- Username: **testuser**
- Password: **mpcaup2030**

Po potvrzení se objeví otázka, zda chcete vyčíst aktuální konfiguraci serveru, zadejte **Ano**. Tím máte server nakonfigurován.

Nyní ověříme funkčnost sdílení dat. Spusťte

12.2. Přihlášení do MES systému a popis rozhraní

K webové aplikaci pro správu činnosti MES systému je možné se připojit prostřednictvím prohlížeče na adrese **http://skumes.skupra.cz**. Jako uživatelské jméno zadejte text „pcX“, heslo je pak „studentX“, kde X je číslo vašeho pracoviště.

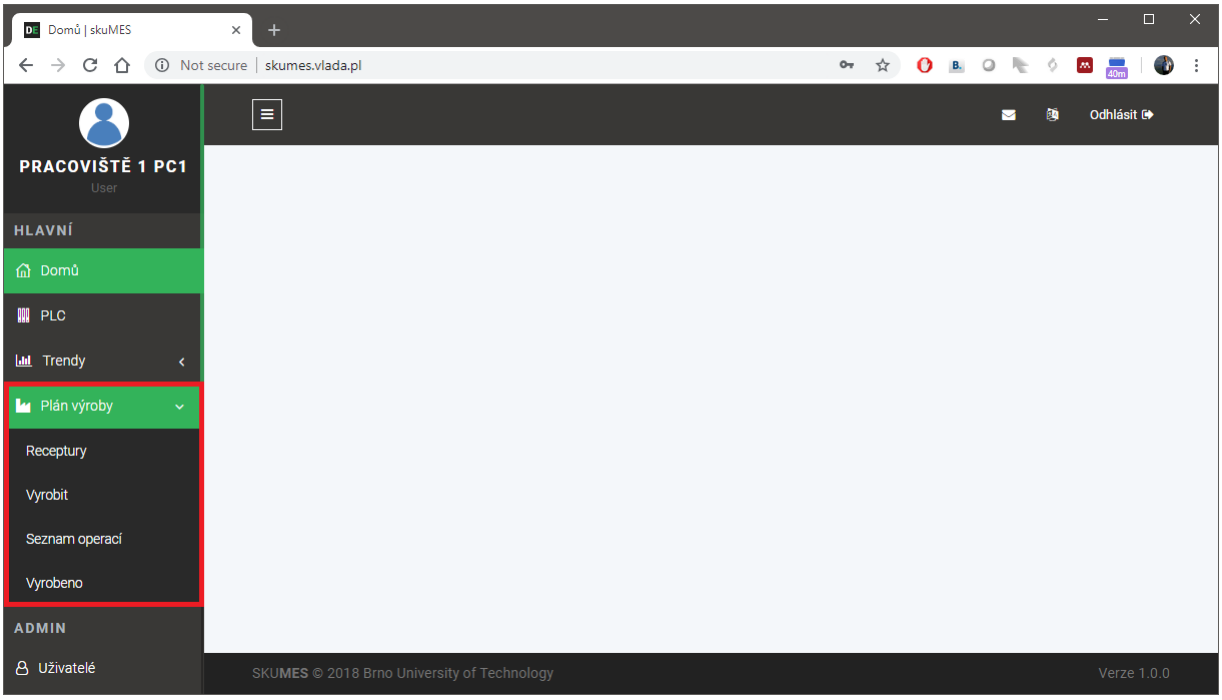


Obr. 7: Přihlášení do aplikace SkuMES

Na Obr. 7 je zobrazeno přihlašovací okno aplikace. Po vyplnění uživatelského jména a hesla stiskněte tlačítko „Přihlásit se“ pro přihlášení. Je možné rovněž vytvoření nového uživatele prostřednictvím odkazu „Registrujte se“. Takový uživatel však má práva pouze pro nahlížení do systému. Nemá možnost tedy nic měnit.

Po přihlášení se dostanete do prostředí SkuMES. SkuMES je modulární systém. V současné době obsahuje pouze dva moduly, z nichž pouze jeden je možné plnohodnotně ovládat prostřednictvím webu.

- SkuMES Plán výroby – jedná se o BATCH modul sloužící k řízení výrobních procesů
- SkuMES Trendy – jedná se o modul sloužící pro vizualizaci trendů sledovaných procesních hodnot (v současné době je implementováno pouze výkonné jádro modulu logující trendy do databáze, komplexní zobrazení trendů není podporováno).



Obrázek 53 Prostředí aplikace SkuMES

12.3. Definice komunikace s PLC

Jádro systému SkuMES komunikuje s PLC (v našem případě Simatic S7-1500) prostřednictvím OPC-UA. Komunikace spočívá v tom, že dochází periodicky k obousměrné výměně definované datové struktury. Tato struktura je tvořena z následujících hierarchicky uspořádaných entit:

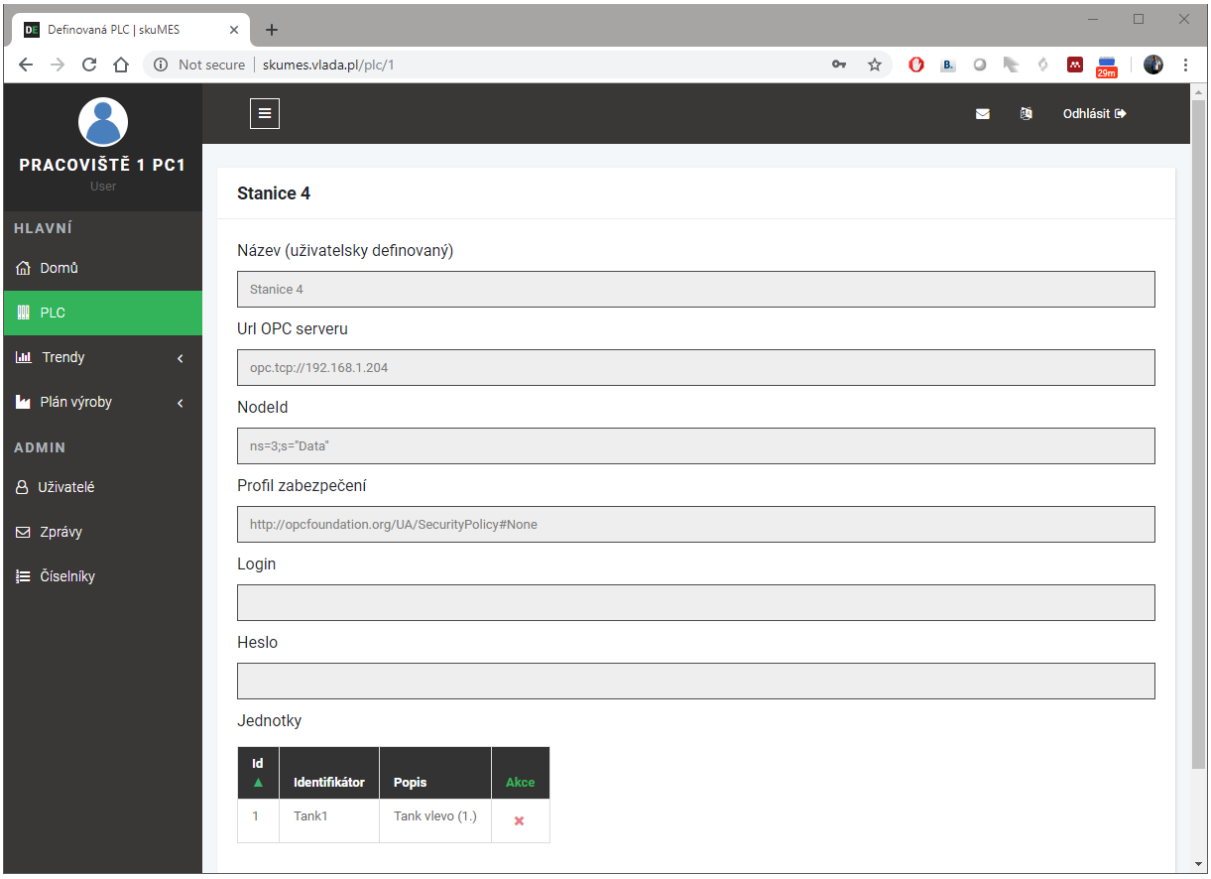
- PLC (Procesní buňka)
    - Jednotka
        - Modul zařízení („fáze“)

- Cmd
- State
- Param

Datová struktura vám je jistě povědomá. Jedná se samozřejmě o strukturu, kterou znáte z PLC programu Tanky, a kterou jsme pro vás připravili. Nyní víte, proč jsme kladli zvláštní důraz na to, že strukturu nelze modifikovat.

V systému SkuMES je musí být rovněž definována podobná struktura, která pro SkuMES vytváří abstrakci fyzického modelu dle S88. Tuto strukturu jsme pro vás už také připravili. Vy ji můžete zkontrolovat prostřednictvím webového rozhraní výběrem položky „PLC“ z nabídky „Hlavní“, následně zvolíte příslušnou stanici (PLC) (viz Obr. 9).

Na tomto obrázku si můžete všimnout nastavení přístupu k OPC serveru stanice, tedy jeho URL, NodeId (což je prefix, který určuje konkrétní část objektové hierarchie v serveru – všimněte si zejména textu „Data“, tedy názvu datového bloku v PLC programu). Dále se zde nachází konfigurace profilu zabezpečení dle specifikace OPC Foundation, a v případě požadavku na zabezpečené připojení i login a heslo k PLC.

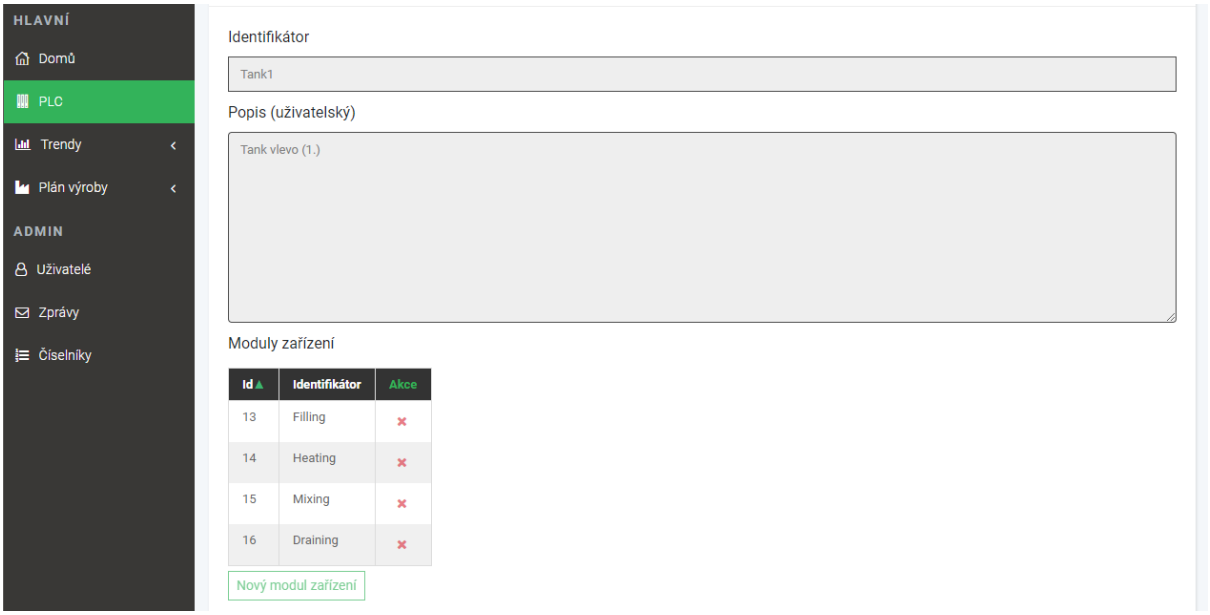


Obr. 9: Nastavení PLC komunikace

V tabulce „Jednotky“ se pak nachází seznam všech definovaných jednotek v procesní buňce. V našem případě se jedná pouze o jediný tank (Tank 1).

Kliknutím na příslušný řádek jednotky se otevře pohled s detailními informacemi o jednotce (Obr. 10). Zde lze definovat popis jednotky a dále její moduly zařízení[1]. Tento krok tedy definuje strukturu modulů zařízení ve fyzickém zařízení (viz Obr. 2). Jména modulů zařízení v jednotce musí přesně korespondovat se jmény modulů zařízení definovanými v datovém bloku PLC.

Kromě jména však není pro moduly zařízení nutno nic nastavovat, neboť struktura datového bloku v PLC programu, která slouží pro výměnu (synchronizaci) dat se předpokládá známá a neměnná.



Obr. 10: Nastavení konkrétní jednotky

Mapování řízení na zařízení určuje hranice mezi řízeními. Jedno z možných řešení, které je využito právě v systému SkuMES, je mapování fází na řídících modulech. Základní řízení se tedy vykonává v PLC a procedurální řízení zajišťuje jádro systému BATCH. Při návrhu základního řízení v PLC musíme respektovat požadavky výrobce BATCH systému. Ve vašem projektu se tak stalo právě proto, že jste použili předpřipravné datové bloky a implementovali pouze výkonný kód.

12.4. Tvorba receptur a jejich životní cyklus

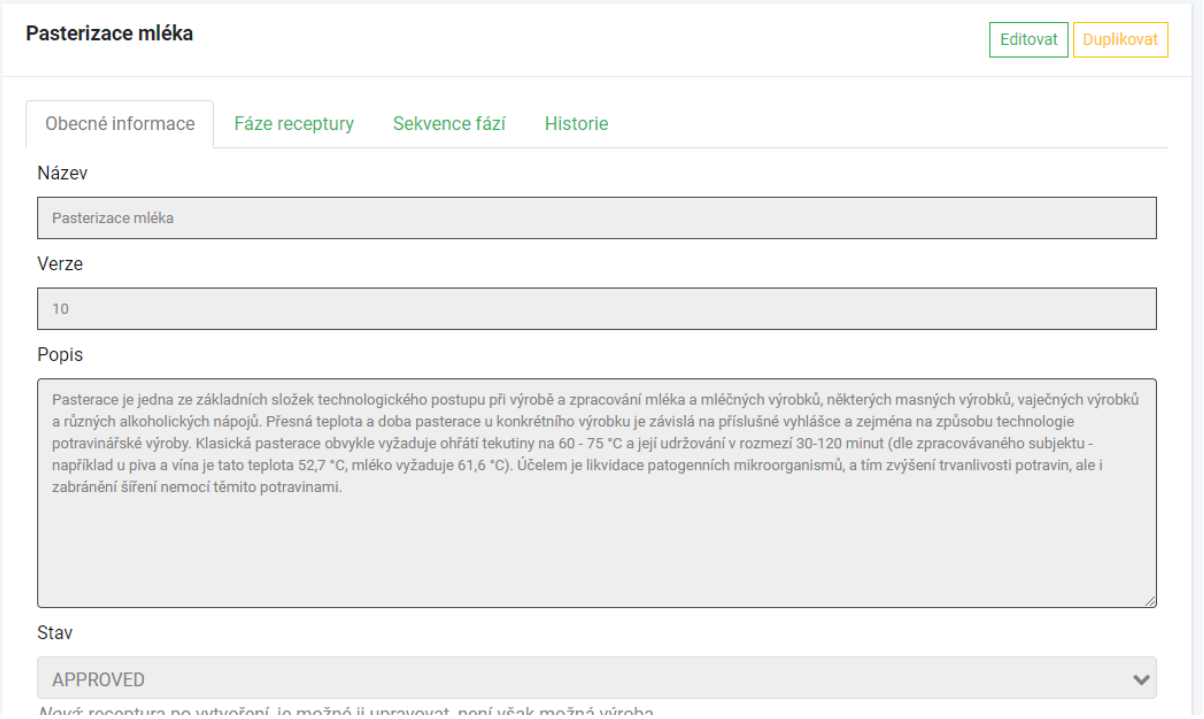
Základním stavebním prvkem procedurálního řízení je, jak už víte, procedura jako součást receptury. Prakticky je možné, aby byly receptury definovány buď v rámci MES systému nebo v jiném systému (například systému PLM, pokud jej firma využívá). V PLM systému bude mít firma nad recepturami lepší kontrolu, bude možné evidovat nejen receptury jako takové, ale rovněž jejich vztah k jednotlivým aspektům obchodního řetězce apod.

V našem případě však receptury definujeme v systému MES. V záložce Plán výroby – Receptury je možné zobrazit seznam definovaných receptur. Každá z receptur v seznamu je identifikována svým názvem, popisem a stavem.

[1] Někdy je nesprávně označujeme jako „fáze“ právě proto, že v našem způsobu implementace figurují na stejné úrovni ve fyzickém modelu, jako fáze v modelu procedurálním.

Po vytvoření se receptura nachází ve stavu **Nová**. V tomto stavu je možné ji parametrizovat. Po schválení receptury se její stav změní na **Schválená**. Teprve nyní je možné na základě této receptury vyrábět dávky. Naopak, recepturu v tomto stavu nelze již nijak modifikovat. To má samozřejmě svůj smysl – ve chvíli, kdy jsou na základě této receptury vyrobeny nějaké dávky, chceme jako výrobce navždy evidovat, jakým způsobem dávky vznikly.

Potřebujeme-li provést úpravu schválené receptury, jedinou možností je recepturu duplikovat. V takovém případě se vytvoří nová receptura ve stavu **Nová**. Aby bylo možné snáze odlišit takto duplikované receptury, má každá receptura po svém vytvoření verzi 1. Každou duplikací se verze receptury zvyšuje. U každé vyrobené dávky je samozřejmě evidováno, na základě které receptury a které verze byl produkt vyroben. Podíváme-li se na Obr. 11, uvidíme detaily receptury pasterizace mléka, kterou budete využívat v rámci cvičení. Vidíte nyní, jakým způsobem lze asi recepturu duplikovat.



Obr. 11: Obecné informace o receptuře

Jedinou možností změny schválené receptury je její převod do stavu **Archivovaná**. U takových receptur se již nepředpokládá výroba – není možné je pro výrobu zvolit. Je samozřejmě možné vyrobit duplikát takové receptury stejně, jako bylo popsáno výše.

Pozn.: Ne všichni uživatelé systému mají možnost vytvářet-mazat-duplikovat-schvalovat receptury. Jednotlivé akce se nastavují pro konkrétní uživatelé separátně.

Každá receptura má, mimo svou hlavičku, kterou jsme viděli na Obr. 11 také ještě *Formuli*, která v rámci tohoto MESu není explicitně definována[2] a samozřejmě *proceduru*. Procedura receptury se rozpadá na jednotkové procedury, operace a fáze. Vzhledem k tomu, že náš software je vyvinut pro řízení výroby na jednoduchých technologiích, vymezují se zatím naše možnosti pouze na definování sekvence nejnižších procedurálních entit - fází. Předpokládáme zde, že během výroby zvolíme procesní

[2] Neřešíme zde proces nakládání se zásobami ani další záležitosti, se kterými by definování formule dávalo smysl.

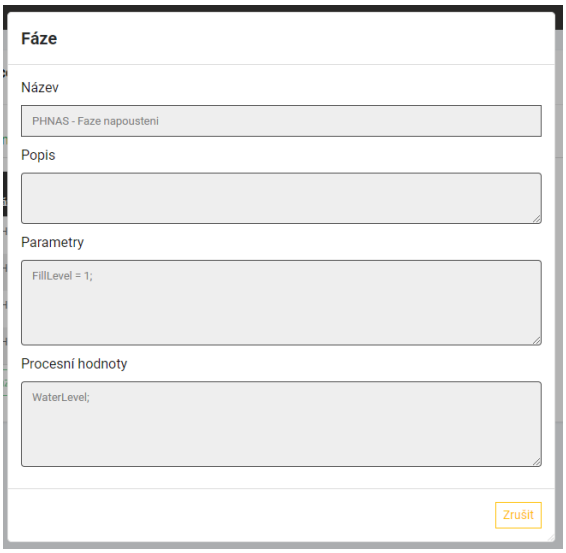
buňku, a dále jednotku, na které bude probíhat výroba. Ta bude realizována v rámci jediné operace (protože je jediná, není nijak explicitně specifikována) pomocí sekvence fází.

Na kartě Fáze receptury se tedy definuje seznam fází (zatím bez jakékoli návaznosti na sebe). Každá z fází receptury má svůj název, volitelný popis a dále seznam parametrů a procesních hodnot. Seznam parametrů se vkládá ve formátu **Název=Hodnota;<nový řádek>**, přičemž dovolené jsou pouze parametry definované v datovém objektu modulu zařízení v podsložce **Param**. Procesní hodnoty se zapisují jako **Název1;<nový řádek>Název2;** atd… Dovolené jsou pouze takové názvy, které jsou definované přímo v rootu datového objektu modulu zařízení. Pro lepší názornost se podívejte na Obr. 12 a

Nastavené parametry jsou před spuštěním konkrétní fáze kopírovány do modulu zařízení. Ve chvíli, kdy je konkrétní fáze spuštěna, jsou periodicky vyčítány a ukládány do databáze definované procesní hodnoty. Tyto hodnoty mohou být později využity pro generování reportů z výroby.

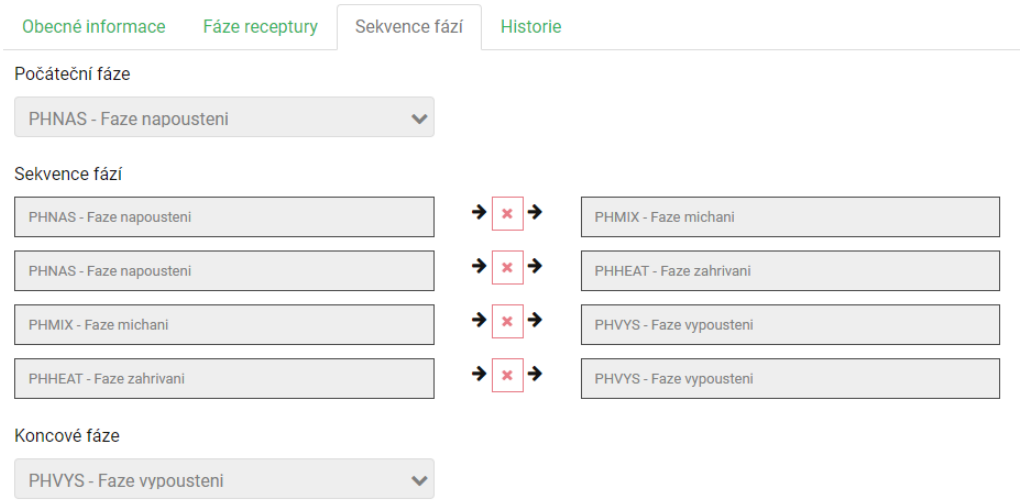
8		Filling	"PhaseFilling"		☐	☑	☑	☑	☐
9		Cmd	Struct		☐	☑	☑	☑	☐
10		State	Struct		☐	☑	☑	☑	☐
11		Param	Struct		☐	☑	☑	☑	☐
12		FillLevel	Real	10.0	☐	☑	☑	☑	☐
13		WaterLevel	Real	0.0	☐	☑	☑	☑	☐

Obr. 12: Parametry a procesní hodnoty fáze napouštění v datovém bloku modulu zařízení



Obr. 13: Konfigurace názvu, parametrů a procesních hodnot ve fázi procedury

Víme tedy nyní, jakým způsobem definujeme jednotlivé fáze nějaké receptury. Nyní je na místě, abychom určili, v jakém pořadí se tyto fáze budou v rámci receptury vyskytovat, potažmo vykonávat. Na záložce Sekvence fází můžete vidět totéž, co na Obr. 14.



Obr. 14: Sekvence fází procedury receptury pasterizace mléka

Způsob definování se jeví jako poněkud chaotický. Je to z důvodu jednoduchosti implementace konfigurační aplikace. Popsaný obrázek tedy říká, že

1) Počáteční fází je fáze napouštění. Tuto fázi bude BATCH spouštět jako první při startu výroby dávky.

2) Po dokončení fáze napouštění se tok řízení rozdělí a budou se vykonávat dvě další fáze souběžně – ZAHŘÍVÁNÍ a MÍCHÁNÍ.

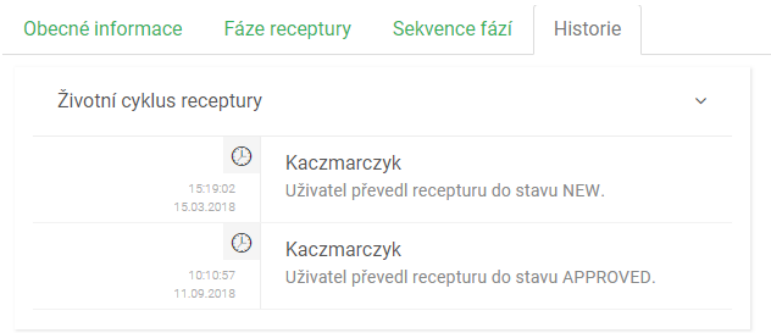
3) Jakmile budou ukončeny obě dvě tyto fáze, tok řízení se opět spojí do fáze VYPOUŠTĚNÍ.

4) Fáze vypouštění bude fází, kterou vykonávání dávky končí.

Pozn.: Je nutné vyplnit počáteční a koncovou fázi!

Pozn.: Při definování vlastní sekvence fází buďte opatrní, program neřeší nekorektní zadání, zacyklení toku apod!

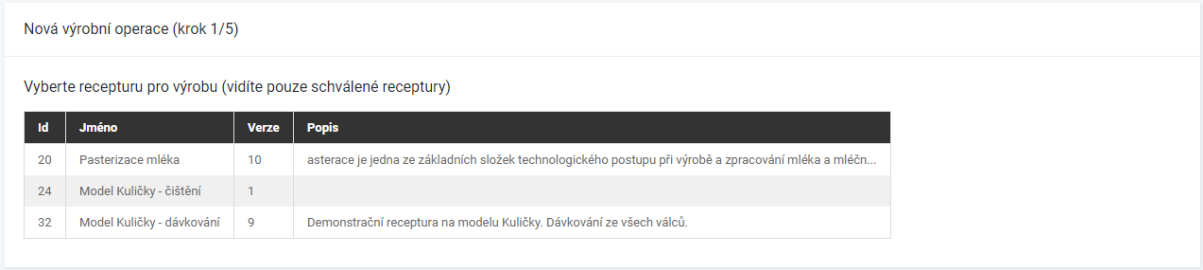
Systém SkuMES sleduje na velice jednoduché bázi životní cyklus každé z receptur. Ve chvíli, kdy je konkrétní receptura převedena uživatelem do některého z výše popsaných stavů, je tato skutečnost zaznamenána v databázi. V záložce Historie příslušné receptury lze následně dohledat, kdo, kdy a do jakého stavu recepturu přepnul.



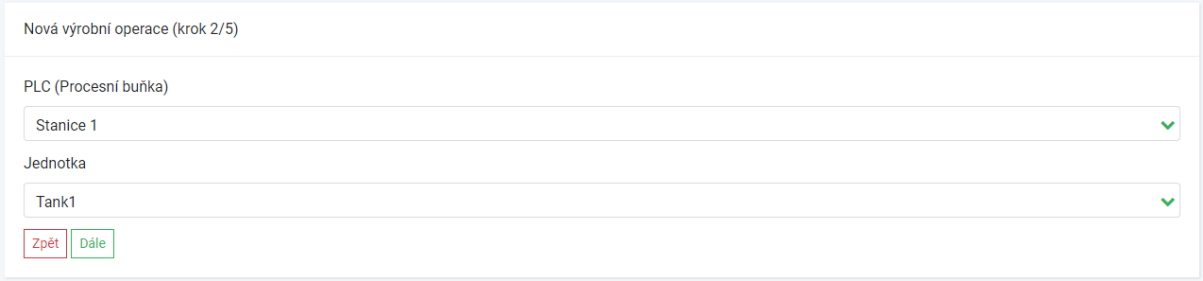
Tím jsme vyčerpali možnosti definování receptur v systému SkuMES. Jen podotkněme, že takto definovaná receptura je **hlavní recepturou**. Z ní budou před výrobou každé dávky vytvářeny klony – **řídicí receptury** specifické pro každou dávku.

12.5. Naplánování výroby

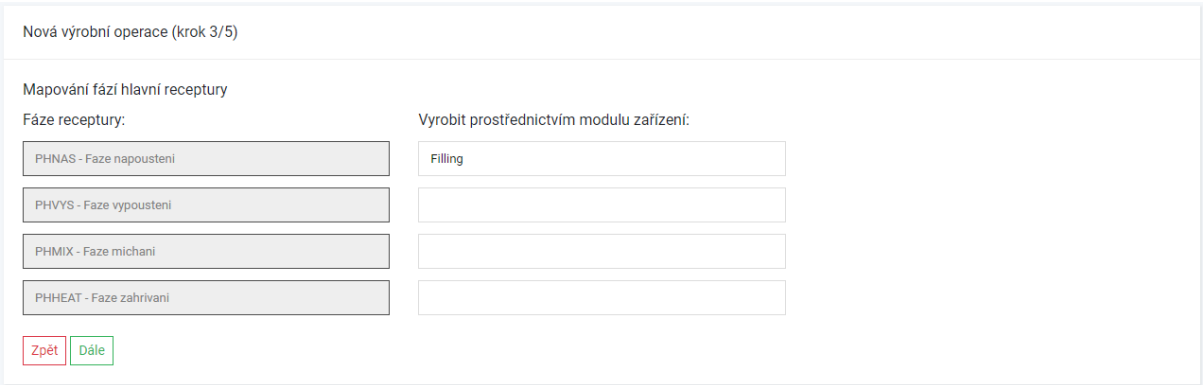
Nyní je na čase vyrobit podle naší receptury dávku. Volbou Plán výroby – Vyrobit z levé nabídky se dostanete do sekce, ve které definujete parametry této dávky. V prvním kroku je potřeba vybrat konkrétní recepturu pro výrobu. Vyberte nyní tedy recepturu pasterizace mléka.



V dalším kroku je potřeba vybrat procesní buňku a konkrétní jednotku, na které bude výroba dávky probíhat. Vyberte tedy své PLC a jednotku Tank1. Stiskněte tlačítko **Dále**.



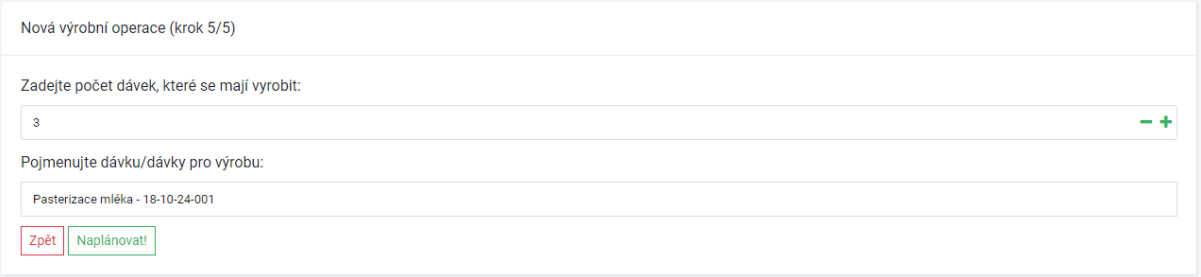
Nyní přicházíme ke kroku, ve kterém přiřadíme jednotlivé fáze procedury receptury ke konkrétním modulům zařízení. Zde dejte pozor, program vás nijak neupozorní, pokud na fázi napouštění namapujete například modul zařízení míchání – nemá jak – BATCH systém netuší, jaká konkrétní funkcionalita se za jednotlivými moduly zařízení skrývá. Všechny mají stejné rozhraní a BATCH je tedy od sebe nemůže nijak rozlišit (což je mimochodem také účelem). Po namapování všech fází stiskněte tlačítko **Dále**.



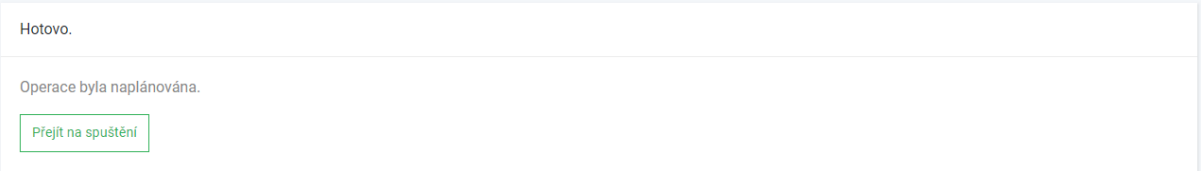
Výrobu lze parametrizovat jednak řazením jednotlivých fází procedury hlavní receptury, jednak nastavením parametrů pro tyto fáze. Existuje však možnost, jak parametry této procedury (mírně) modifikovat, například při kolísání nějaké vlastnosti některého ze vstupních materiálů. Proto je při výrobě konkrétní dávky vytvářen klon hlavní receptury – receptura řídicí. V této řídicí receptuře již nemůžeme ovlivnit pořadí fází, ale je možné provést změnu parametrů. V okně, které nyní vidíte před sebou, vidíte seznam všech parametrů všech fází procedury hlavní receptury. Můžete nyní provést změnu některého z parametrů. Po nastavení stiskněte tlačítko **Dále**.



Posledním krokem je specifikovat, kolik dávek má být vyrobeno s konfigurací, kterou jste nyní provedli. Zadejte tedy příslušný počet (v systému je nastaveno omezení na vytvoření max. 5 dávek). Svou dávku (či dávky) také smysluplně pojmenujte tak, abyste je dokázali odlišit od dávek vytvořených na jiném pracovišti. Nakonec stiskněte tlačítko **Naplánovat!**



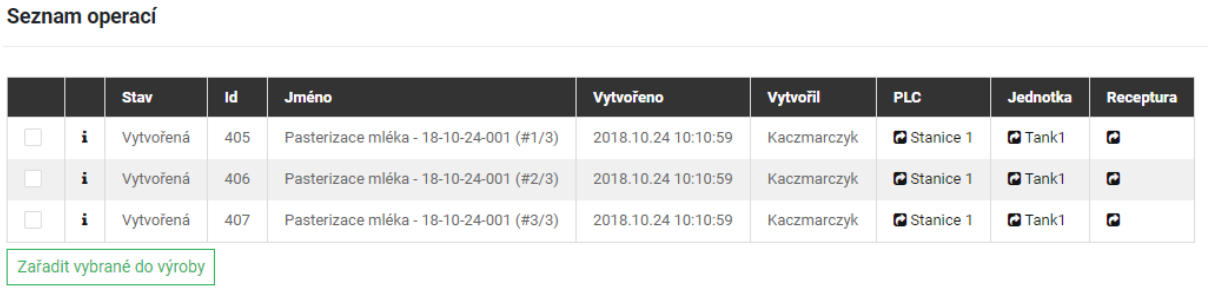
Tím je tedy plánování dokončeno. Stiskem tlačítka **Přejít na spuštění** se dostanete do formuláře **Seznam operací**.



12.6. Spuštění plánovaných operací

Na stránce **Vyrobit – Seznam operací** vidíte seznam všech dávek, které ještě nebyly vyrobeny a jejich aktuální stav. Můžete si všimnout, že ve chvíli, kdy vytvoříte více dávek hromadně, jsou jednotlivé dávky odlišeny přidáním indexu na konec jejich jména. Z každé dávky můžete odkazem přejít na specifikaci konkrétní procesní buňky, jednotky a příslušné receptury. Stiskem tlačítka se pak

dostanete k detailům výroby konkrétní dávky. Můžete vidět, že v případě dávky ve stavu Vytvořená nejsou k dispozici žádné detaily.



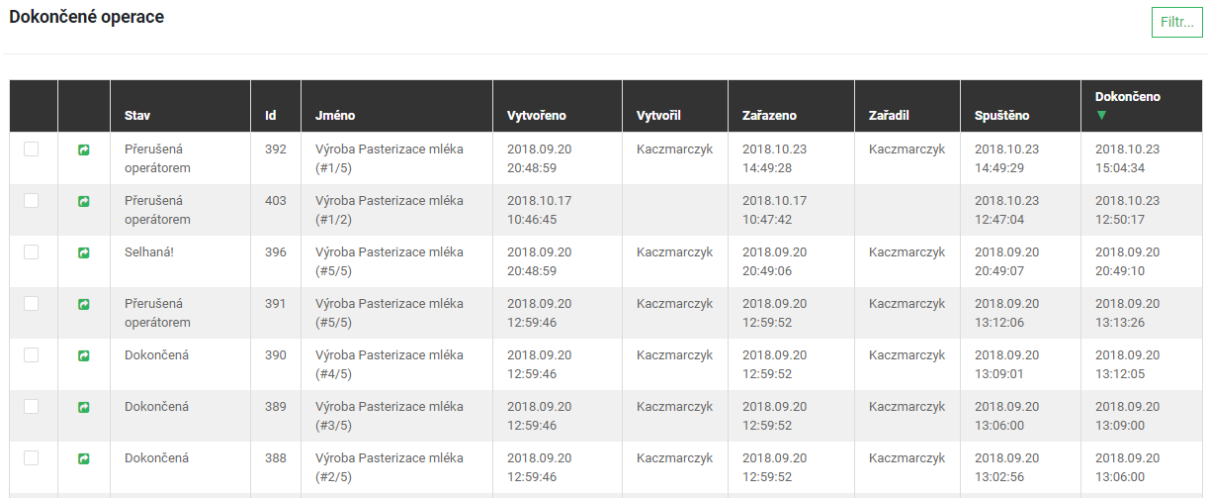
Výrobu konkrétní dávky/dávek spustíte tak, že změníte stav zaškrtávače na příslušném řádku na ☑ a stisknete tlačítko **Zařadit vybrané do výroby**.

12.7. Sledování průběhu výroby

Sledujte!

12.8. Report z výroby

Dokončené dávky zobrazíte výběrem **Plán výroby – Vyrobeno** z hlavní nabídky. V tabulce jsou uvedeny všechny dávky, které již byly vyrobeny, nebo jejichž výroba nějakým způsobem selhala. Klepnutím na ikonu u příslušné dávky můžete přejít na report z výroby.



Report následně obsahuje několik sekcí:

- Obecné informace – zejména časové údaje a osoby zodpovědné za plánování a výrobu, dále pak detailně identifikuje použitou hlavní recepturu
- Parametry řídicí receptury – které mohly být nastaveny jinak, než tomu je u receptury hlavní

- Podrobný průběh – seznam startu a ukončení jednotlivých fází výroby na modulech zařízení. V této sekci je rovněž tlačítko, pomocí kterého lze exportovat do souboru .csv časový průběh sledovaných procesních hodnot.
- Podpis
- Štítky s čárovými kódy – lze jimi označit např. zásobník, do kterého byl hotový produkt přemístěn.

Vytiskněte si report z výroby vaší dávky. Bude sloužit jako příloha k dokumentaci.